Универзитет у Београду
Факултет организационих наука
Катедра за софтверско инжењерство



Семинарски рад из предмета
Пројектовање софтвера

Софтверски систем за управљање ветеринарским услугама у Јава окружењу

Ментор: **проф. др Синиша Влајић**

Студент: Николија Станковић
Број индекса: 2019/19

Београд, 2026.

Садржај

1. Увод 3
2. Прикупљање корисничких захтева 4
2.1 Вербални опис 4
2.1 Случајеви коришћења 6
3. Анализа 16
3.1 Понашање софтверског система - Одређивање системских операција на основу сценарија случаја коришћења 16
3.2 Понашање софтверског система- Секвенцни дијаграми случаја коришћења 18
3.3 Понашање софтверског система - Дефинисање уговора о системским операцијама 28
3.4 Структура софтверског система – Концептуални (доменски) модел 30
3.6 Табела структурних и вредносних ограничења релационог модела 32
4. Пројектовање 35
4.1 Пројектовање корисничког интерфејса 36
4.2 Пројектовање екранских форми 37
4.3 Пројектовање контролера корисничког интерфејса 59
4.4 Пројектовање апликационе логике 59
4.4.1 Контролер апликационе логике 59
4.5 Пословна логика 60
4.5.1 Пројектовање понашања софтверског система (системске операције) 60
4.5.2 Пројектовање структуре софтверског система 65
4.5.3 Пројектовање складишта података 72
5. Имплементација 76
6. Тестирање 80
7. Закључак 82
Литература 83

1. Увод

У савременом добу, убрзани развој информационих технологија и процес дигитализације намећу потребу за ефикасним софтверским системима који унапређују рад организација и све више задовољавају захтеве корисника. Задатак софтверских инжењера је да креирају системе који су поуздани, флексибилни и прилагодљиви, а истовремено једноставни за употребу. Развој софтвера стога подразумева добро осмишљен процес који обезбеђује поуздан крајњи производ прилагођен стварним потребама.

У овом раду биће приказан процес развоја софтверског система за праћење рада ветеринарске ординације. Циљ система је да омогући ефикасно управљање терминима између клијената и ветеринара, као и обраду и складиштење података о услугама. На почетку ће бити представљена упрошћена Ларманова метода развоја софтвера, која је дала значајан допринос у области организације и управљања софтверским пројектима, наглашавајући важност корисничких захтева и ефикасности система.

Као програмски језик за реализацију пројекта изабрана је Јава, која се током више деценија наметнула као један од најпопуларнијих и најпоузданијих алата за развој софтверских апликација. Њена објектно-оријентисана структура, могућност извршавања на више платформи и аутоматско управљање меморијом чине је погодном за развој комплексних система. Поред тога, једноставна синтакса и безбедно руковање типовима података знатно олакшавају процес програмирања.

У другом поглављу биће описан процес прикупљања корисничких захтева, и то кроз вербални опис система као и идентификацију случајева коришћења. На тај начин ће бити постављена основа за разумевање потреба и очекивања крајњих корисника ветеринарске ординације, тачније, клијената.

Треће поглавље биће посвећено анализи система. У оквиру ове фазе биће разматрано понашање софтверског система на основу сценарија случајева коришћења, израђени секвенцни дијаграми и дефинисани уговори о системским операцијама. Поред тога, биће представљен концептуални модел домена, релациони модел и табела структурних и вредносних ограничења релационог модела.

Четврто поглавље односиће се на пројектовање система, које обухвата дизајн корисничког интерфејса, пројектовање апликационе логике и складишта података. Овим делом рада поставља се темељ за прелазак у фазу реализације.

Пето поглавље обухвата имплементацију развијеног решења у Јава програмском језику, док ће у шестом бити детаљно описано тестирање система кроз проверу исправности и поузданости свих функционалности. У седмом поглављу биће изложен закључак са осврtом на добијене резултате, изазове са којима смо се током рада сусретали, као и предности предложеног решења. На крају, биће представљена литература коришћена током израде рада.

2. Прикупљање корисничких захтева

2.1 Вербални опис

Потребно је направити Софтверски систем за управљање ветеринарским услугама у Јава окружењу. Софтверски систем, односно његова пословна логика (у општем смислу), састоји се из следећих *апстрактних концепата*:
а) **пружалац услуге**
б) **прималац услуге**
ц) **документ** који описује процес пружања услуге
д) **шифарници** у којима се налазе подаци о конкретним концептима који се користе у процесу пружања услуге, а који нису пружалац услуге, прималац услуге или документ који описује процес пружања услуге.
У наведеном софтверском систему *пружалац услуге* је Ветеринар, *прималац услуге* је Клијент, *документ* који описује процес пружања услуге је Термин. *Шифарници* су Категорија клијента, Услуга, Лиценца.

Конкретни концепти су између себе повезани на следећи начин:
Преко једног термина је могуће заказати више услуга. Термин је повезан са једним ветеринаром и једним клијентом. Клијент је везан за једну Категорију клијента. Ветеринар може бити везан за више лиценци, док лиценца може бити везана за више ветеринара.

При пријављивању на софтверски систем потребно је обезбедити ***аутентификацију*** (преко корисничког имена и шифре) корисника софтверског система.
Потребно је обезбедити следеће функционалности за наведене конкретне концепте:

Редни број концепта	**Концепт**	**Функционалности**
1.	Термин	Убаци, Претражи, Промени
2.	Клијент	Убаци, Претражи, Промени, Обриши
3.	Ветеринар	Пријави, Убаци, Претражи, Промени, Обриши
4.	Услуга	Убаци, Претражи, Промени, Обриши
5.	Категорија клијента	Убаци, Претражи, Промени, Обриши
6	Лиценца	Убаци, Претражи, Промени, Обриши
7.	Ставка термина	
8.	Ветеринар-Лиценца	

Табела 1: Повезивање концепата и функционалности

Функционалностима софтверског система се приступа преко главног менија који има следећу структуру:

1. Документи
    1.1 Термин
2. Пружалац услуге
    2.1 Ветеринар
3. Прималац услуге
    3.1 Клијент
4. Шифарници
    4.1 Категорија клијента,
    4.2 Услуга,
    4.3 Лиценца,
5. Подешавања софтверског система
6. О програму

2.1 Случајеви коришћења

На основу наведених функционалности концепата уочени су следећи случајеви коришћења:

Шифра случаја коришћења	*Назив случаја коришћења*	*Предуслови С.К. (листе)*	*Критеријуми претраживања се односе на:*
SK1	**СК1- Убаци термин**	**Учитане су листе: a) Ветеринар b) Клијент c) Услуга**	
SK2	**СК2- Претражи термин**		**a) Термин b) Ветеринар c) Клијент d) Услуга**
SK3	**СК3- Промени термин**	**Учитане су листе: a) Ветеринар b) Клијент c) Услуга**	**a) Термин b) Ветеринар c) Клијент d) Услуга**
SK4	**СК4- Убаци клијента**	**Учитане су листе: a) Категорија клијента**	
SK5	**СК5- Претражи клијента**		**a) Клијент b) Категорија клијента**
SK6	**СК6- Промени клијента**	**Учитане су листе: a) Категорија клијента**	**a) Клијент b) Категорија клијента**
SK7	**СК7- Обриши клијента**		**a) Клијент b) Категорија клијента**
SK8	**СК8- Пријави ветеринара**		
SK9	**СК9- Убаци ветеринара**	**Учитане су листе: a) Лиценца**	
SK10	**СК10- Претражи ветеринара**		**a) Ветеринар b) Лиценца**
SK11	**СК11- Промени ветеринара**	**Учитане су листе: a) Лиценца**	**a) Ветеринар b) Лиценца**
SK12	**СК12- Обриши ветеринара**		**a) Ветеринар b) Лиценца**
SK13	**СК13- Убаци услугу**		
SK14	**СК14- Претражи услугу**		**a) Услуга**
SK15	**СК15- Промени услугу**		**a) Услуга**
SK16	**СК16- Обриши услугу**		**a) Услуга**
SK17	**СК17- Убаци категорију клијента**		
SK18	**СК18- Претражи категорију клијента**		**a) Категорија клијента**
SK19	**СК19- Промени категорију клијента**		**a) Категорија клијента**
SK20	**СК20- Обриши категорију клијента**		**a) Категорија клијента**
SK21	**СК21- Убаци лиценцу**		
SK22	**СК22- Претражи лиценцу**		**a) Лиценца**
SK23	**СК23- Промени лиценцу**		**a) Лиценца**
SK24	**СК24- Обриши лиценцу**		**a) Лиценца**

Табела 2: Случајеви коришћења софтверског система

За следеће случајеве коришћења ћемо дати детаљан опис:

СК1- Убаци термин
СК2- Претражи термин
СК3- Промени термин
СК4- Убаци клијент
СК5- Претражи клијент
СК6- Промени клијент
СК7- Обриши клијент
СК8- Пријави ветеринар

Табела 3: Случајеви коришења за које ће бити дат детаљан опис

СК1- Убаци термин

Назив СК
Убаци термин

Актори СК
Ветеринар

Учесници СК
Ветеринар, кориснички интерфејс (клијентски програм) и систем (серверски програм)

Предуслови: Кориснички интерфејс (клијентски програм) и систем (серверски програм) су покренути. Ветеринар је пријављен под својом шифром. Систем приказује форму за рад са термином. *Учитане су листе: a) Ветеринар b) Клијент c) Услуга*

Основни сценарио СК:

1. Ветеринар **уноси** податке о термину. (АПУСО)
2. Ветеринар **контролише** да ли је коректно унео податке о термину. (АНСО)
3. Ветеринар **позива** систем да запамти податке о термину. (АПСО)
4. Систем **памти** податке о термину. (СО)
5. Систем **приказује** ветеринару поруку: "Систем је запамтио термин." (ИА)

Алтернативна сценарија:

5.1 Уколико систем не може да запамти податке о термину он **приказује** ветеринару поруку: "Систем не може да запамти термин". (ИА)

СК2- Претражи термин

Назив СК
Претражи термин

Актори СК
Ветеринар

Учесници СК
Ветеринар, кориснички интерфејс (клијентски програм) и систем (серверски програм)

Предуслови: Кориснички интерфејс (клијентски програм) и систем (серверски програм) су покренути. Ветеринар је пријављен под својом шифром. Кориснички интерфејс приказује форму за рад са термином. На наведеној екранској форми су дефинисани *критеријуми,* који се односе на: a) Термин b) Ветеринар c) Клијент d) Услуга, који ће да врате листу термина.

Основни сценарио СК:

1. Ветеринар **бира** критеријуме на основу којих претражује термине. (АПУСО)
2. Ветеринар **позива** систем да нађе термине по задатим критеријумима. (АПСО)
3. Систем **тражи** термине по задатим критеријумима. (СО)
4. Систем **приказује** ветеринару термине и поруку: “Систем је нашао термине по задатим критеријумима”. (ИА)
5. Ветеринар **бира** термин. (АПУСО)
6. Ветеринар **позива** систем да нађе термин. (АПСО)
7. Систем **тражи** термин. (СО)
8. Систем **приказује** ветеринару термин и поруку: “Систем је нашао термин”. (ИА)

Алтернативна сценарија:

4.1 Уколико систем не може да нађе термине он **приказује** ветеринару поруку: “Систем не може да нађе термине по задатим критеријумима”. Прекида се извршење сценарија. (ИА)
8.1 Уколико систем не може да нађе термин он **приказује** ветеринару поруку: “Систем не може да нађе термин”. (ИА)

СК3- Промени термин

Назив СК
Промени термин

Актори СК
Ветеринар

Учесници СК
Ветеринар, кориснички интерфејс (клијентски програм) и систем (серверски програм)

Предуслови: Кориснички интерфејс (клијентски програм) и систем (серверски програм) су покренути. Ветеринар је пријављен под својом шифром. Кориснички интерфејс приказује форму за рад са термином. На наведеној екранској форми су дефинисани *критеријуми,* који се односе на: a) Термин b) Ветеринар c) Клијент d) Услуга, који ће да врате листу термина. Учитане су листе: a) Ветеринар b) Клијент c) Услуга

Основни сценарио СК:

1. Ветеринар **бира** критеријуме на основу којих претражује термине. (АПУСО)
2. Ветеринар **позива** систем да нађе термине по задатим критеријумима. (АПСО)
3. Систем **тражи** термине по задатим критеријумима. (СО)
4. Систем **приказује** ветеринару термине и поруку: “Систем је нашао термине по задатим критеријумима”. (ИА)
5. Ветеринар **бира** термин. (АПУСО)
6. Ветеринар **позива** систем да нађе термин. (АПСО)
7. Систем **тражи** термин. (СО)
8. Систем **приказује** ветеринару термин и поруку: “Систем је нашао термин”. (ИА)
9. Ветеринар **уноси (мења)** податке о термину. (АПУСО)
10. Ветеринар **контролише** да ли је коректно унео податке о термину. (АНСО)
11. Ветеринар **позива** систем да запамти податке о термину. (АПСО)
12. Систем **памти** податке о термину. (СО)
13. Систем **приказује** ветеринару поруку: “Систем је запамтио термин.” (ИА)

Алтернативна сценарија:

4.1 Уколико систем не може да нађе термине он **приказује** ветеринару поруку: “Систем не може да нађе термине по задатим критеријумима”. Прекида се извршење сценарија. (ИА)
8.1 Уколико систем не може да нађе термин он **приказује** ветеринару поруку: “Систем не може да нађе термин”. Прекида се извршење сценариа. (ИА)
13.1 Уколико систем не може да запамти податке о термину он **приказује** ветеринару поруку: “Систем не може да запамти термин”. (ИА)

СК4- Убаци клијент

Назив СК
Убаци клијент

Актори СК
Ветеринар

Учесници СК
Ветеринар, кориснички интерфејс (клијентски програм) и систем (серверски програм)

Предуслови: Кориснички интерфејс (клијентски програм) и систем (серверски програм) су покренути. Ветеринар је пријављен под својом шифром. Систем приказује форму за рад са клијентом. *Учитане су листе: а) Категорија клијента*

Основни сценарио СК:

1. Ветеринар **уноси** податке о клијенту. (АПУСО)
2. Ветеринар **контролише** да ли је коректно унео податке о клијенту. (АНСО)
3. Ветеринар **позива** систем да запамти податке о клијенту. (АПСО)
4. Систем **памти** податке о клијенту. (СО)
5. Систем **приказује** ветеринару поруку: "Систем је запамтио клијента." (ИА)

Алтернативна сценарија:

5.1 Уколико систем не може да запамти податке о клијенту он **приказује** ветеринару поруку: "Систем не може да запамти клијента". (ИА)

СК5- Претражи клијент

Назив СК
Претражи клијент

Актори СК
Ветеринар

Учесници СК
Ветеринар, кориснички интерфејс (клијентски програм) и систем (серверски програм)

Предуслови: Кориснички интерфејс (клијентски програм) и систем (серверски програм) су покренути. Ветеринар је пријављен под својом шифром. Кориснички интерфејс приказује форму за рад са клијентом. На наведеној екранској форми су дефинисани *критеријуми,* који се односе на: a) Клијент b) Категорија клијента, који ће да врате листу клијената.

Основни сценарио СК:

1. Ветеринар **бира** критеријуме на основу којих претражује клијенте. (АПУСО)
2. Ветеринар **позива** систем да нађе клијенте по задатим критеријумима. (АПСО)
3. Систем **тражи** клијенте по задатим критеријумима. (СО)
4. Систем **приказује** ветеринару клијенте и поруку: "Систем је нашао клијенте по задатим критеријумима". (ИА)
5. Ветеринар **бира** клијента. (АПУСО)
6. Ветеринар **позива** систем да нађе клијента. (АПСО)
7. Систем **тражи** клијента. (СО)
8. Систем **приказује** ветеринару клијента и поруку: "Систем је нашао клијента". (ИА)

Алтернативна сценарија:

4.1 Уколико систем не може да нађе клијенте он **приказује** ветеринару поруку: "Систем не може да нађе клијенте по задатим критеријумима". Прекида се извршење сценарија. (ИА)
8.1 Уколико систем не може да нађе клијента он **приказује** ветеринару поруку: "Систем не може да нађе клијента". (ИА)

СК6- Промени клијент

Назив СК
Промени клијент

Актори СК
Ветеринар

Учесници СК
Ветеринар, кориснички интерфејс (клијентски програм) и систем (серверски програм)

Предуслови: Кориснички интерфејс (клијентски програм) и систем (серверски програм) су покренути. Ветеринар је пријављен под својом шифром. Кориснички интерфејс приказује форму за рад са клијентом. На наведеној екранској форми су дефинисани *критеријуми,* који се односе на: a) Клијент b) Категорија клијента, који ће да врате листу клијената. Учитане су листе: a) Категорија клијента

Основни сценарио СК:

1. Ветеринар **бира** критеријуме на основу којих претражује клијенте. (АПУСО)
2. Ветеринар **позива** систем да нађе клијенте по задатим критеријумима. (АПСО)
3. Систем **тражи** клијенте по задатим критеријумима. (СО)
4. Систем **приказује** ветеринару клијенте и поруку: "Систем је нашао клијенте по задатим критеријумима". (ИА)
5. Ветеринар **бира** клијента. (АПУСО)
6. Ветеринар **позива** систем да нађе клијента. (АПСО)
7. Систем **тражи** клијента. (СО)
8. Систем **приказује** ветеринару клијента и поруку: "Систем је нашао клијента". (ИА)
9. Ветеринар **уноси (мења)** податке о клијенту. (АПУСО)
10. Ветеринар **контролише** да ли је коректно унео податке о клијенту. (АНСО)
11. Ветеринар **позива** систем да запамти податке о клијенту. (АПСО)
12. Систем **памти** податке о клијенту. (СО)
13. Систем **приказује** ветеринару поруку: "Систем је запамтио клијента." (ИА)

Алтернативна сценарија:

4.1 Уколико систем не може да нађе клијенте он **приказује** ветеринару поруку: "Систем не може да нађе клијенте по задатим критеријумима". Прекида се извршење сценарија. (ИА)
8.1 Уколико систем не може да нађе клијента он **приказује** ветеринару поруку: "Систем не може да нађе клијента". Прекида се извршење сценариа. (ИА)
13.1 Уколико систем не може да запамти податке о клијенту он **приказује** ветеринару поруку: "Систем не може да запамти клијента". (ИА)

СК7- Обриши клијент

Назив СК
Обриши клијент

Актори СК
Ветеринар

Учесници СК
Ветеринар, кориснички интерфејс (клијентски програм) и систем (серверски програм)

Предуслови: Кориснички интерфејс (клијентски програм) и систем (серверски програм) су покренути. Ветеринар је пријављен под својом шифром. Кориснички интерфејс приказује форму за рад са клијентом. На наведеној екранској форми су дефинисани *критеријуми,* који се односе на: a) Клијент b) Категорија клијента, који ће да врате листу клијената.

Основни сценарио СК:

1. Ветеринар **бира** критеријуме на основу којих претражује клијенте. (АПУСО)
2. Ветеринар **позива** систем да нађе клијенте по задатим критеријумима. (АПСО)
3. Систем **тражи** клијенте по задатим критеријумима. (СО)
4. Систем **приказује** ветеринару клијенте и поруку: “Систем је нашао клијенте по задатим критеријумима”. (ИА)
5. Ветеринар **бира** клијента. (АПУСО)
6. Ветеринар **позива** систем да нађе клијента. (АПСО)
7. Систем **тражи** клијента. (СО)
8. Систем **приказује** ветеринару клијента и поруку: “Систем је нашао клијента”. (ИА)
9. Ветеринар **позива** систем да обрише клијента. (АПСО)
10. Систем **брише** клијента. (СО)
11. Систем **приказује** ветеринару поруку: “Систем је обрисао клијента.” (ИА)

Алтернативна сценарија:

4.1 Уколико систем не може да нађе клијенте он **приказује** ветеринару поруку: “Систем не може да нађе клијенте по задатим критеријумима”. Прекида се извршење сценарија. (ИА)
8.1 Уколико систем не може да нађе клијента он **приказује** ветеринару поруку: “Систем не може да нађе клијента”. Прекида се извршење сценарија. (ИА)
11.1 Уколико систем не може да обрише клијента он **приказује** ветеринару поруку: “Систем не може да обрише клијента”. (ИА)

СК8- Пријави ветеринар

Назив СК
Пријави ветеринар

Актори СК
Ветеринар

Учесници СК
Ветеринар, кориснички интерфејс (клијентски програм) и систем (серверски програм)

Предуслови: Кориснички интерфејс (клијентски програм) и систем (серверски програм) су покренути. Кориснички интерфејс приказује форму за ***пријављивање***.

Основни сценарио СК:

1. Ветеринар **уноси** корисничко име и шифру. (АПУСО)
2. Ветеринар **контролише** да ли је коректно унео корисничко име и шифру. (АНСО)
3. Ветеринар **позива** систем да провери корисничко име и шифру. (АПСО)
4. Систем **проверава** корисничко име и шифру. (СО)
5. Систем **приказује** ветеринару поруку: "*Корисничко име и шифра су исправни.*" (ИА)
6. Кориснички интерфејс **позива** главну форму и мени. (КИПГФМ)

Алтернативна сценарија:

5.1 Уколико систем провером установи да корисничко име и/или шифра нису исправни он **приказује** ветеринару поруку: "*Корисничко име и шифра нису исправни".* (ИА)
6.1 Уколико кориснички интерфејс не може да отвори главну форму и мени **приказује** ветеринару поруку: "*Не може да се отвори главна форма и мени*". (НПГФМ)

Постуслови: *Отворена главна форма и мени.*

3. Анализа

3.1 Понашање софтверског система - Одређивање системских операција на основу сценарија случаја коришћења

На основу наведених случајева коришћења уочене су следеће системске операције:

Шифра случаја коришћења	*Назив случаја коришћења*	**Системске операције**
SK1	**СК1-Убаци термин**	**1. signal UbaciTermin(Termin) 2. signal PromeniTermin(Termin) 3. signal vratiListuSviVeterinar(Lista<Veterinar>) 4. signal vratiListuSviKlijent(Lista<Klijent>) 5. signal vratiListuSviUsluga(Lista<Usluga>)**
SK2	**СК2-Претражи термин**	**1. signal PretraziTermin(Termin) 2. signal vratiListuTermin(kriterijumTermin, Lista<Termin>) 3. signal vratiListuTermin(kriterijumVeterinar, Lista<Termin>) 4. signal vratiListuTermin(kriterijumKlijent, Lista<Termin>) 5. signal vratiListuTermin(kriterijumUsluga, Lista<Termin>)**
SK3	**СК3-Промени термин**	**1. signal PromeniTermin(Termin) 2. signal PretraziTermin(Termin) 3. signal vratiListuSviVeterinar(Lista<Veterinar>) 4. signal vratiListuSviKlijent(Lista<Klijent>) 5. signal vratiListuSviUsluga(Lista<Usluga>) 6. signal vratiListuTermin(kriterijumTermin, Lista<Termin>) 7. signal vratiListuTermin(kriterijumVeterinar, Lista<Termin>) 8. signal vratiListuTermin(kriterijumKlijent, Lista<Termin>) 9. signal vratiListuTermin(kriterijumUsluga, Lista<Termin>)**
SK4	**СК4-Убаци клијент**	**1. signal UbaciKlijent(Klijent) 2. signal PromeniKlijent(Klijent) 3. signal vratiListuSviKategorijaKlijenta(Lista<KategorijaKlijenta>)**
SK5	**СК5-Претражи клијент**	**1. signal PretraziKlijent(Klijent) 2. signal vratiListuKlijent(kriterijumKlijent, Lista<Klijent>) 3. signal vratiListuKlijent(kriterijumKategorijaKlijenta, Lista<Klijent>)**
SK6	**СК6-Промени клијент**	**1. signal PromeniKlijent(Klijent) 2. signal PretraziKlijent(Klijent) 3. signal vratiListuSviKategorijaKlijenta(Lista<KategorijaKlijenta>) 4. signal vratiListuKlijent(kriterijumKlijent, Lista<Klijent>) 5. signal vratiListuKlijent(kriterijumKategorijaKlijenta, Lista<Klijent>)**
SK7	**СК7-Обриши клијент**	**1. signal ObrisiKlijent(Klijent) 2. signal vratiListuKlijent(kriterijumKlijent, Lista<Klijent>) 3. signal vratiListuKlijent(kriterijumKategorijaKlijent, Lista<Klijent>)**
SK8	**СК8-Пријави ветеринар**	**1. signal PrijaviVeterinar(korisnickoIme, sifra)**

Табела 4: Системске операције случаја коришћења

Наводимо све уочене системске операције:

1. signal UbaciKlijent(Klijent)
2. signal UbaciTermin(Termin)
3. signal ObrisiKlijent(Klijent)
4. signal PretraziKlijent(Klijent)
5. signal PretraziTermin(Termin)
6. signal PrijaviVeterinar(korisnickoIme, sifra)
7. signal PromeniKlijent(Klijent)
8. signal PromeniTermin(Termin)
9. signal vratiListuVeterinar(kriterijumVeterinar, Lista<Veterinar>)
10. signal vratiListuVeterinar(kriterijumLicenca, Lista<Veterinar>)
11. signal vratiListuKlijent(kriterijumKlijent, Lista<Klijent>)
12. signal vratiListuKlijent(kriterijumKategorijaKlijenta, Lista<Klijent>)
13. signal vratiListuKategorijaKlijenta(kriterijumKategorijaKlijenta, Lista<KategorijaKlijenta>)
14. signal vratiListuTermin(kriterijumVeterinar, Lista<Termin>)
15. signal vratiListuTermin(kriterijumKlijent, Lista<Termin>)
16. signal vratiListuTermin(kriterijumTermin, Lista<Termin>)
17. signal vratiListuTermin(kriterijumUsluga, Lista<Termin>)
18. signal vratiListuUsluga(kriterijumUsluga, Lista<Usluga>)
19. signal vratiListuLicenca(kriterijumLicenca, Lista<Licenca>)
20. signal vratiListuSviVeterinar(Lista<Veterinar>)
21. signal vratiListuSviKlijent(Lista<Klijent>)
22. signal vratiListuSviKategorijaKlijenta(Lista<KategorijaKlijenta>)
23. signal vratiListuSviUsluga(Lista<Usluga>)
24. signal vratiListuSviLicenca(Lista<Licenca>)

Табела 5: Системске операције софтверског система

3.2 Понашање софтверског система- Секвенцни дијаграми случаја коришћења

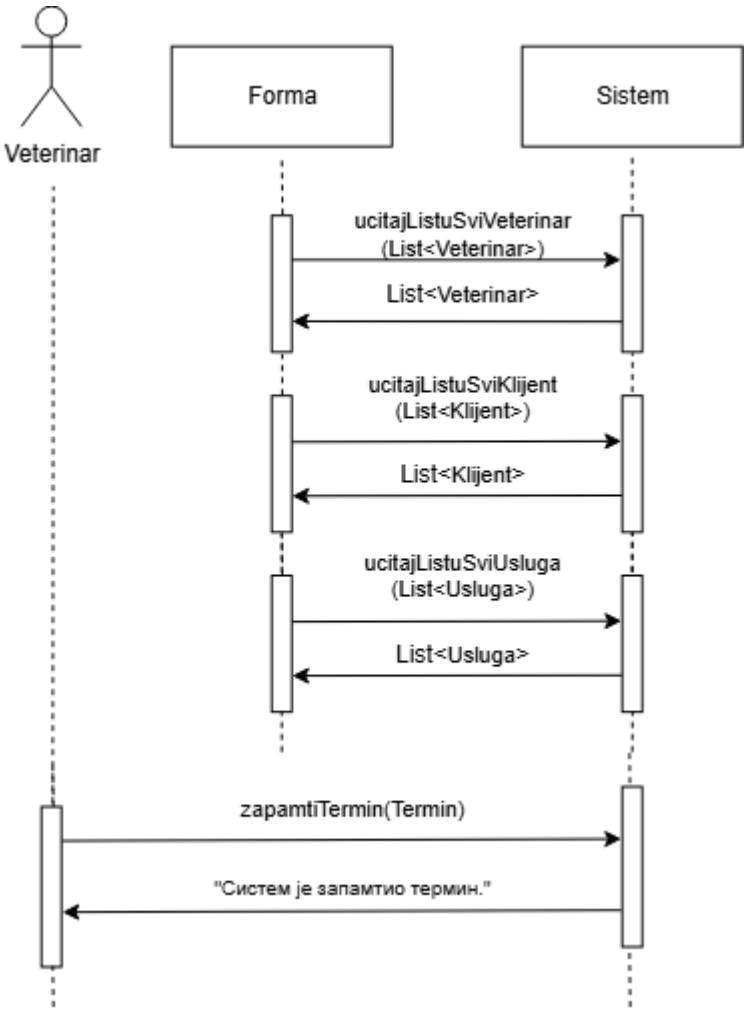
ДС1*: Дијаграми секвенци случаја коришћења – Убаци термин*

Предуслови:

1. Форма **позива** систем да **врати** листу свих ветеринара. (АПСО)
2. Систем **враћа** форми листу свих ветеринара. (ИА)
3. Форма **позива** систем да **врати** листу свих клијената. (АПСО)
4. Систем **враћа** форми листу свих клијената. (ИА)
5. Форма позива систем да врати листу свих услуга. (АПСО)
6. Систем враћа форми листу свих услуга. (ИА)

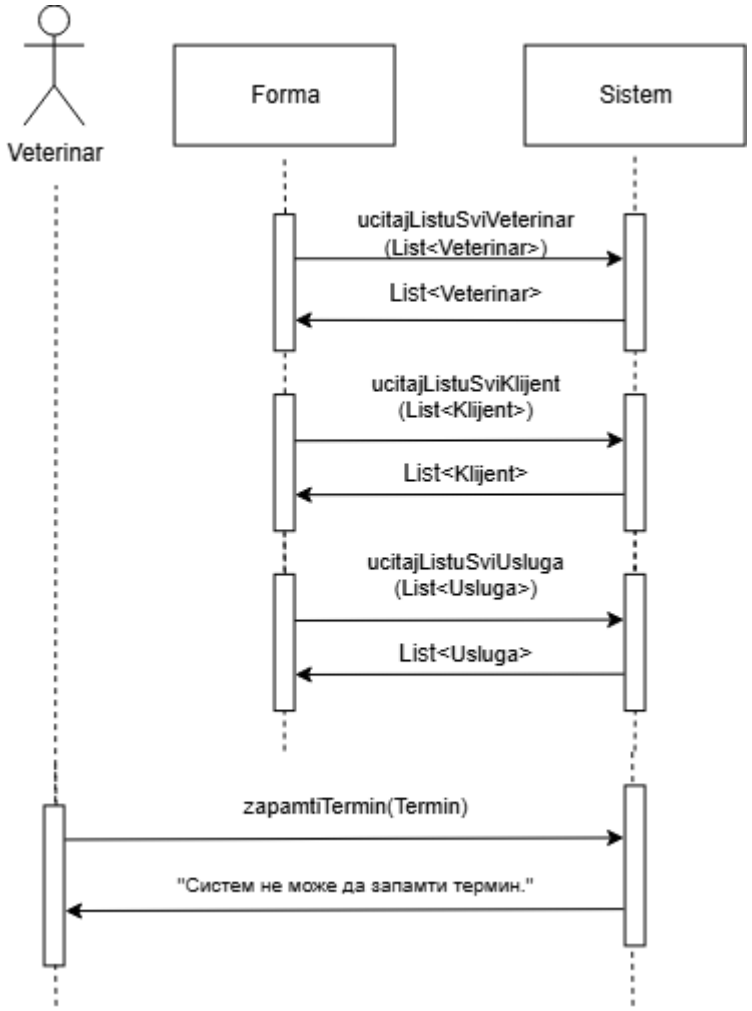
Основни сценарио СК:

7. Ветеринар **позива** систем да запамти податке о термину. (АПСО)
8. Систем **приказује** ветеринару поруку: “Систем је запамтио термин.” (ИА)



Алтернативна сценарија СК:

8.1 Уколико систем не може да запамти податке о термину он **приказује** ветеринару поруку: “Систем не може да запамти термин”. (ИА)



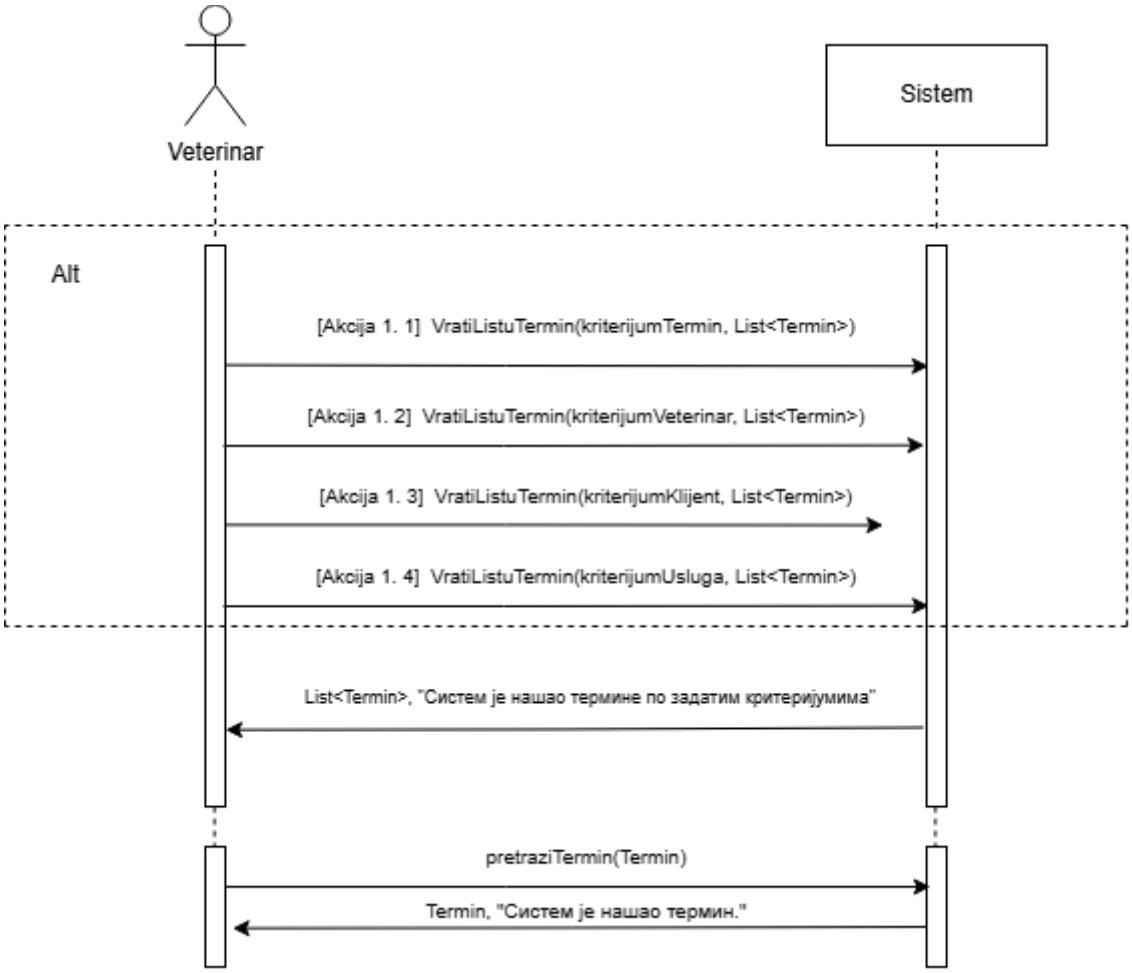
Са наведених секвенцних дијаграма уочавају се 4 системске операције које треба пројектовати:

1. signal UbaciTermin(Termin)
2. signal PromeniTermin(Termin)
3. signal vratiListuSviVeterinar(Lista<Veterinar>)
4. signal vratiListuSviKlijent(Lista<Klijent>)
5. signal vratiListuSviUsluga(Lista<Usluga>)

ДС2: *Дијаграми секвенци случаја коришћења – Претражи термин*

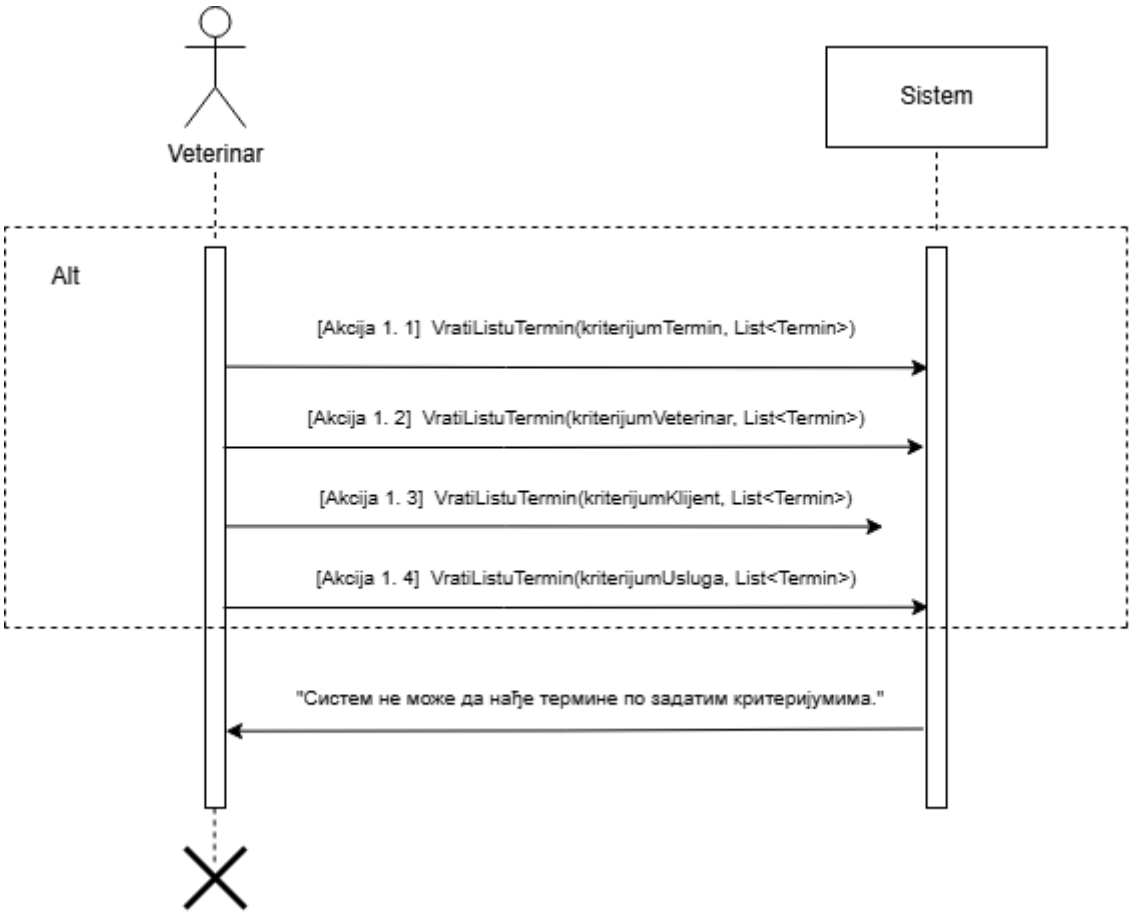
Основни сценарио СК:

1. Ветеринар **позива** систем да нађе термине по задатим критеријумима. (АПСО)
2. Систем **приказује** ветеринару термине и поруку: “Систем је нашао термине по задатим критеријумима”. (ИА)
3. Ветеринар **позива** систем да нађе термин. (АПСО)
4. Систем **приказује** ветеринару термин и поруку: “Систем је нашао термин”. (ИА)

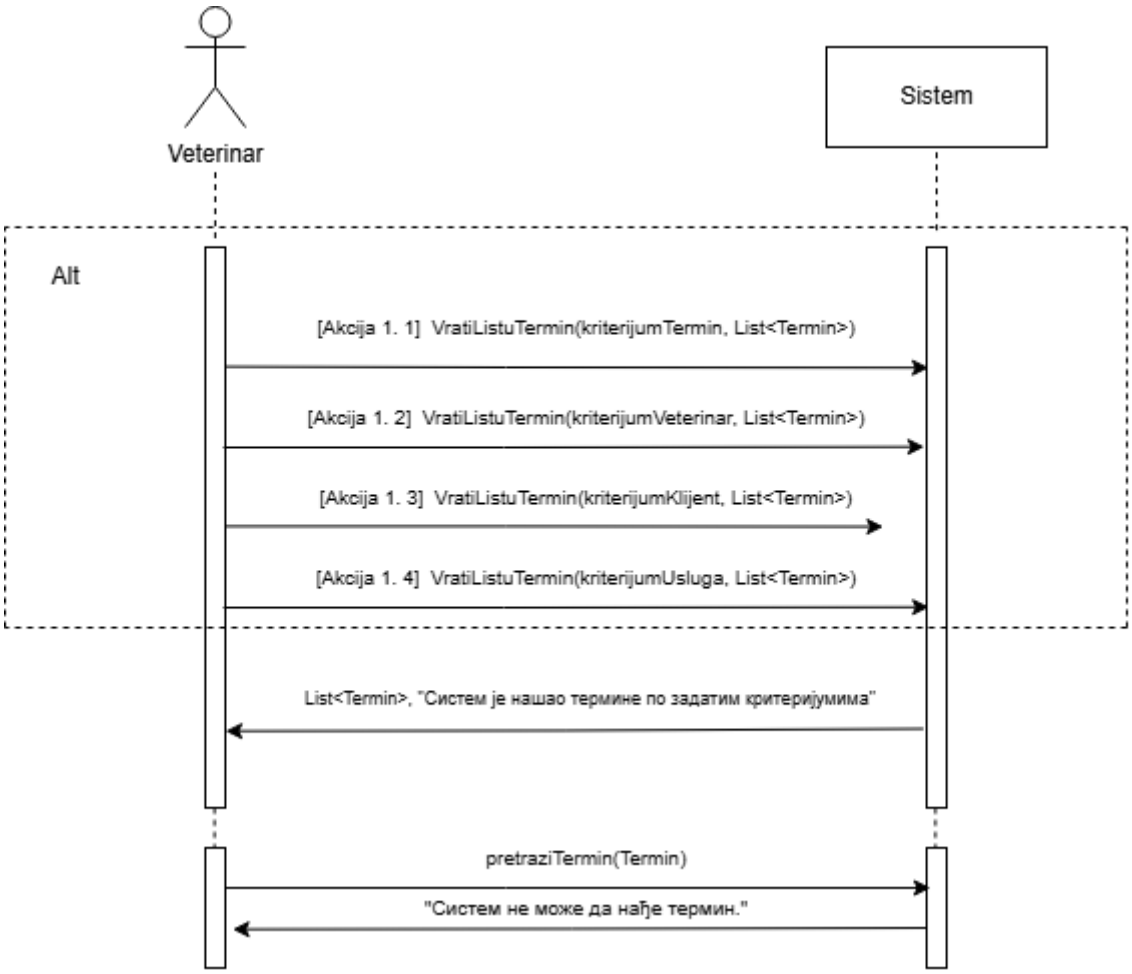


Алтернативна сценарија СК:

2.1 Уколико систем не може да нађе термине он **приказује** ветеринару поруку: “Систем не може да нађе термине по задатим критеријумима”. Прекида се извршење сценарија. (ИА)



4.1 Уколико систем не може да нађе термин он **приказује** ветеринару поруку: “Систем не може да нађе термин”. (ИА)



Са наведених секвенцних дијаграма уочавају се 5 системских операција које треба пројектовати:

1. signal PretraziTermin(Termin)
2. signal vratiListuTermin(kriterijumTermin,Lista<Termin>)
3. signal vratiListuTermin(kriterijumVeterinar,Lista <Termin>)
4. signal vratiListuTermin(kriterijumKlijent,Lista<Termin>)
5. signal vratiListuTermin(kriterijumUsluga,Lista<Termin>)

ДС3: *Дијаграми секвенци случаја коришћења – Промени термин*

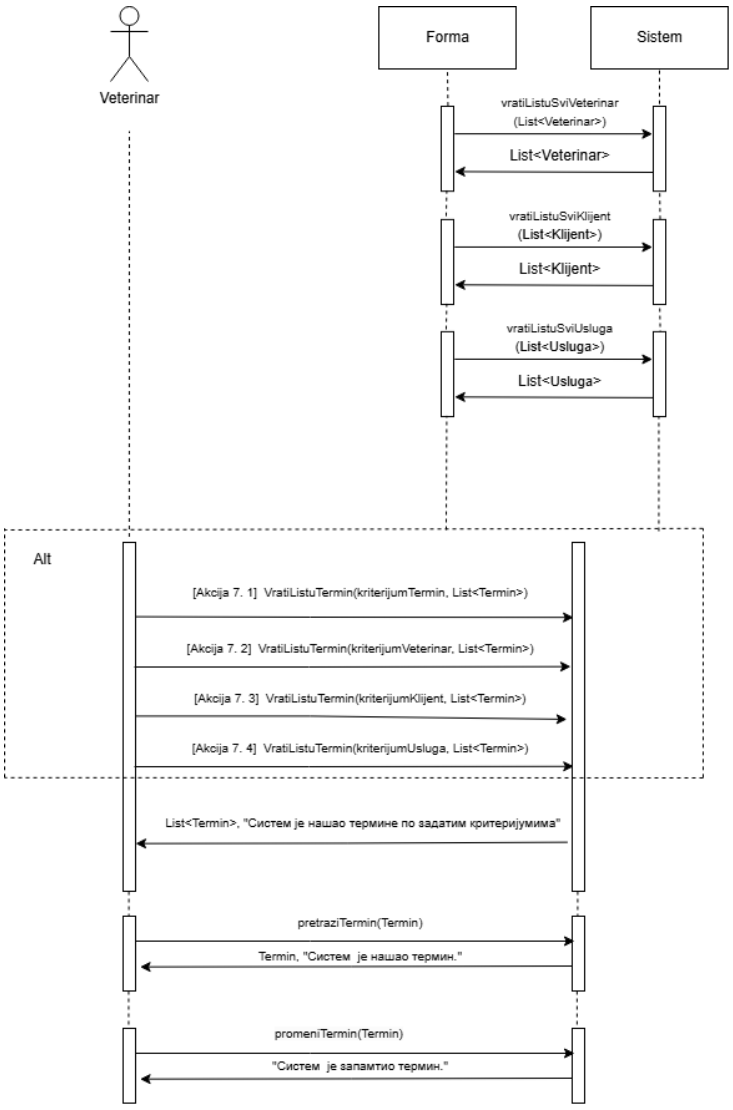
Предуслови:

1. Форма **позива** систем да **врати** листу свих ветеринара. (АПСО)
2. Систем **враћа** форми листу свих ветеринара. (ИА)
3. Форма **позива** систем да **врати** листу свих клијената. (АПСО)
4. Систем **враћа** форми листу свих клијената. (ИА)
5. Форма позива систем да врати листу свих услуга. (АПСО)
6. Систем враћа форми листу свих услуга. (ИА)

Основни сценарио СК:

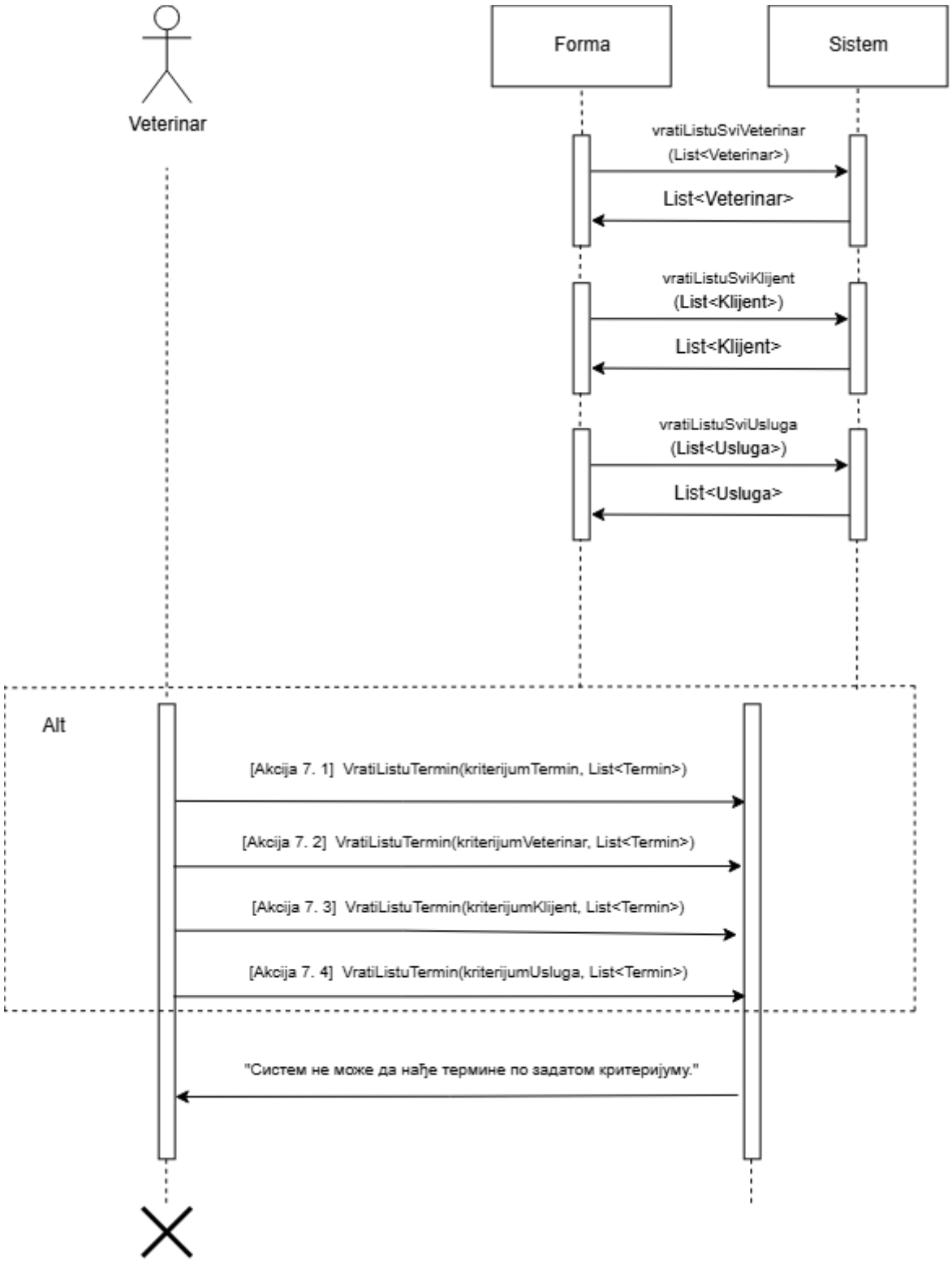
7. Ветеринар **позива** систем да нађе термине по задатим критеријумима. (АПСО)
8. Систем **приказује** ветеринару термине и поруку: “Систем је нашао термине по задатим критеријумима”. (ИА)
9. Ветеринар **позива** систем да нађе термин. (АПСО)
10. Систем **приказује** ветеринару термин и поруку: “Систем је нашао термин”. (ИА)

11. Ветеринар **позива** систем да запамти податке о термину. (АПСО)
12. Систем **приказује** ветеринару поруку: “Систем је запамтио термин.” (ИА)

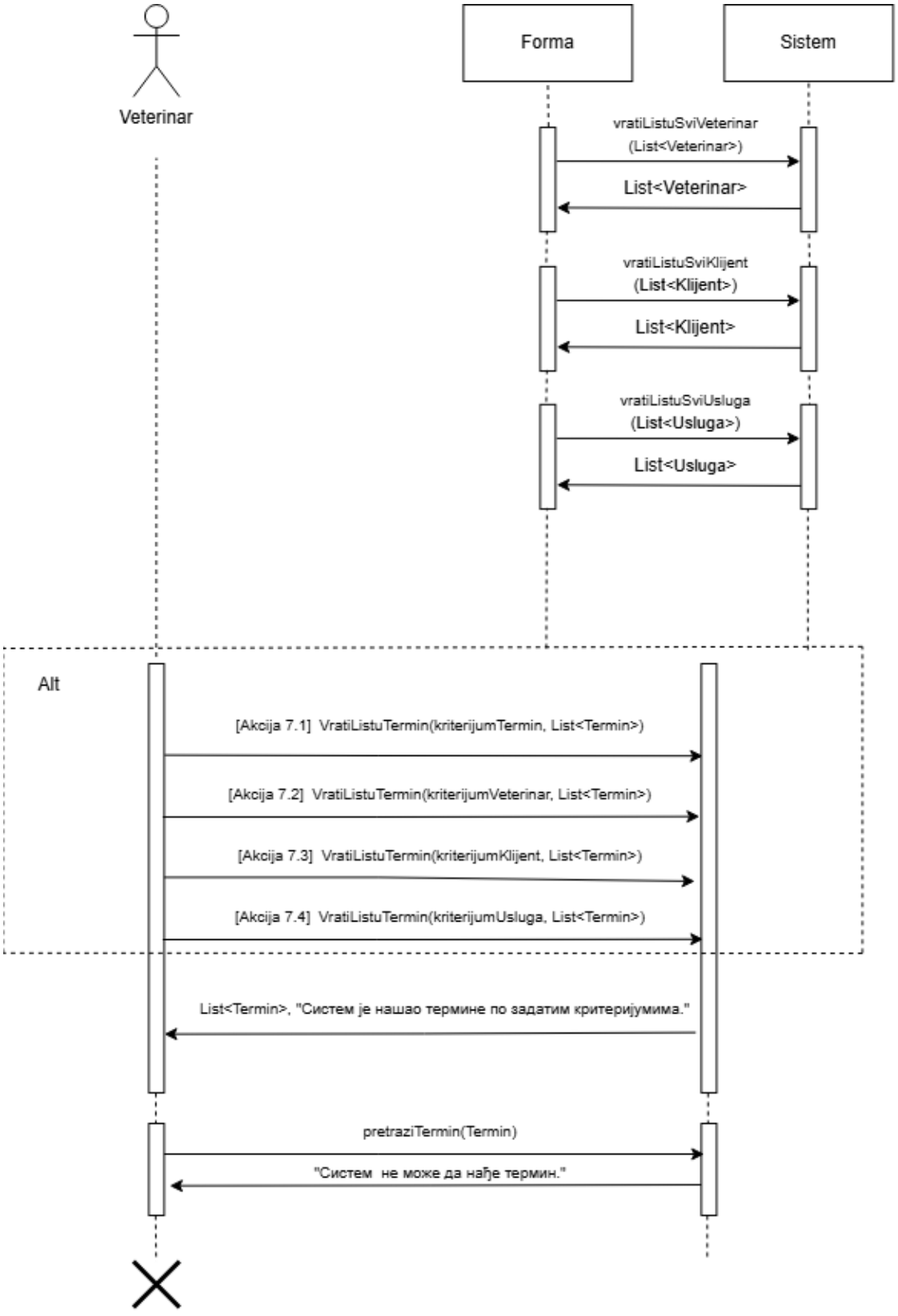


Алтернативна сценарија СК:

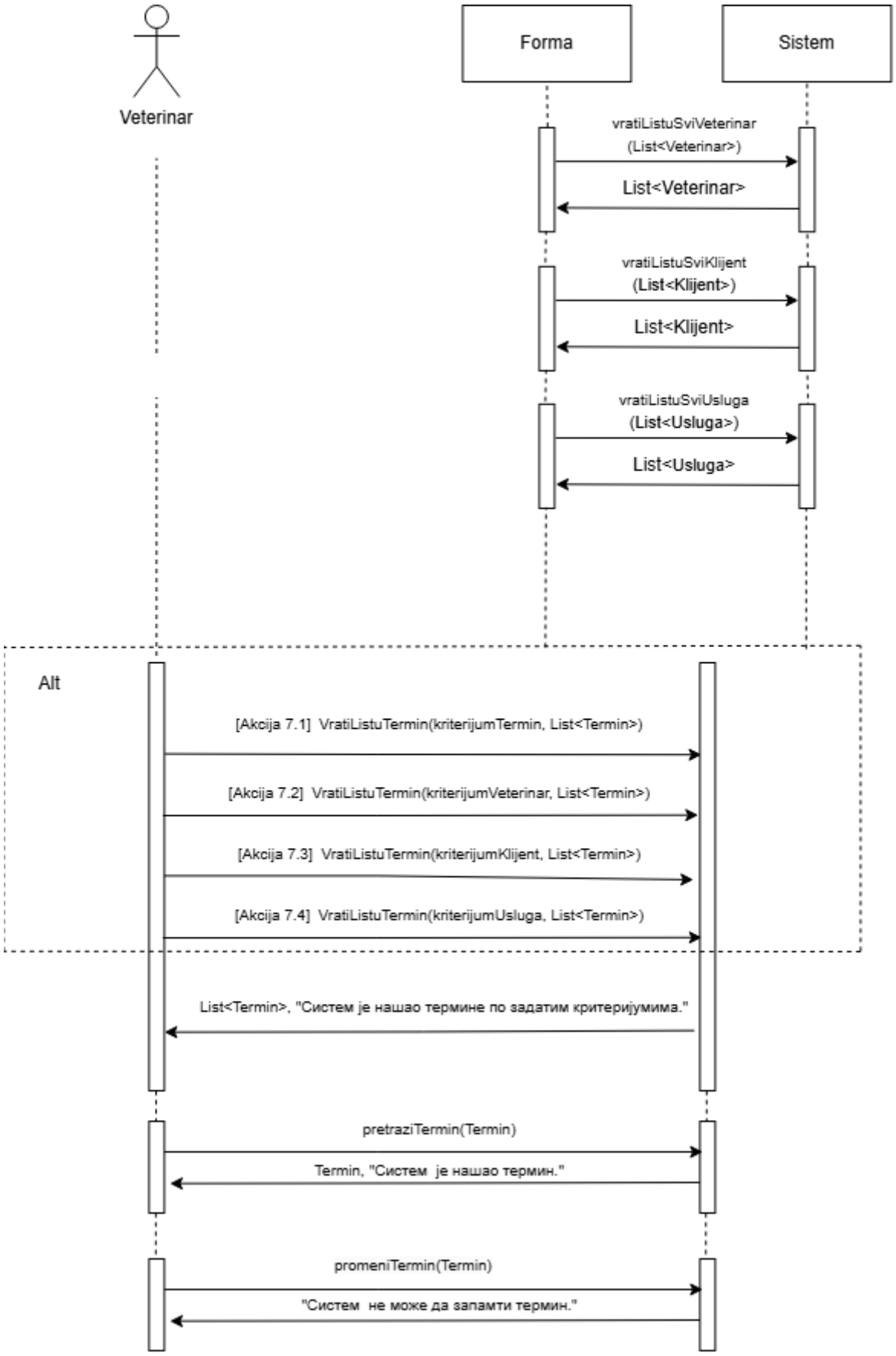
8.1 Уколико систем не може да нађе термине он приказује ветеринару поруку: “Систем не може да нађе термине по задатом критеријуму”. Прекида се извршење сценарија. (ИА)



10.1 Уколико систем не може да нађе термин он **приказује** ветеринару поруку: “Систем не може да нађе термин”. Прекида се извршење сценарија. (ИА)



12.1 Уколико систем не може да запамти податке о термину он **приказује** ветеринару поруку: “Систем не може да запамти термин”. (ИА)



Са наведених секвенцних дијаграма уочавају се 9 системских операција које треба пројектовати:

1.signal zapamtiTermin(Termin)
2.signal pretraziTermin(Termin)
3. signal vratiListuSviVeterinar(Lista<Veterinar>)
4. signal vratiListuSviKlijent(Lista<Klijent>)
5. signal vratiListuSviUsluga(Lista<Usluga>)
6. signal vratiListuTermin(kriterijumTermin, Lista<Termin>)
7. signal vratiListuTermin(kriterijumVeterinar, Lista<Termin>)
8. signal vratiListuTermin(kriterijumKlijent, Lista<Termin>)
9. signal vratiListuTermin(kriterijumUsluga, Lista<Termin>)

3.3 Понашање софтверског система- Дефинисање уговора о системским операцијама

За системске операције се праве уговори. Овде ћемо навести осам различитих (типских) уговора за системске операције.

1. Уговор UG1: PrijaviVeterinar

Операција: PrijavaVeterinar (*Korisnickolme, Sifra*):signal;
Веза са СК: СК8
Предуслови:
Постуслови: *Ветеринар је пријављен на систем.*

2. Уговор UG2: UbaciTermin

Операција: *UbaciTermin(Termin)*:signal;
Веза са СК: СК1
Предуслови: *Структурна и вредносна ограничење над објектом класе Термин и Ставка термина морају бити задовољена.*
Постуслови: *Направљен је нови објекат класе Термин.*

3. Уговор UG3: UbaciKlijent

Операција *UbaciKlijent(Klijent)*:signal;
Веза са СК: СК4
Предуслови: *Структурна и вредносна ограничење над објектом класе Клијент морају бити задовољена.*
Постуслови: *Направљен је нови објекат класе Клијент.*

4. Уговор UG4: PromeniKlijent

Операција: *PromeniKlijent(Klijent)*:signal;
Веза са СК: СК4, СК6
Предуслови: *Структурна и вредносна ограничење над објектом класе Клијент морају бити задовољена.*
Постуслови: *Објекат класе Клијент је промењен.*

5. Уговор UG5: ObrisiKlijent

Операција: *ObrisiKlijent(Klijent*):signal;
Веза са СК: СК7
Предуслови *Структурна ограничења над објектом класе Клијент морају бити задовољена.*
Постуслови: *Објекат класе Клијент је обрисан.*

6. Уговор UG6: PretraziKlijent

Операција: ***PretraziKlijent(Klijent)***:signal;
Веза са СК: СК5, СК6, СК7
Предуслови:
Постуслови: *Пронађен је тражени објекат класе Клијент*.

7. Уговор UG7: vratiListuTermin

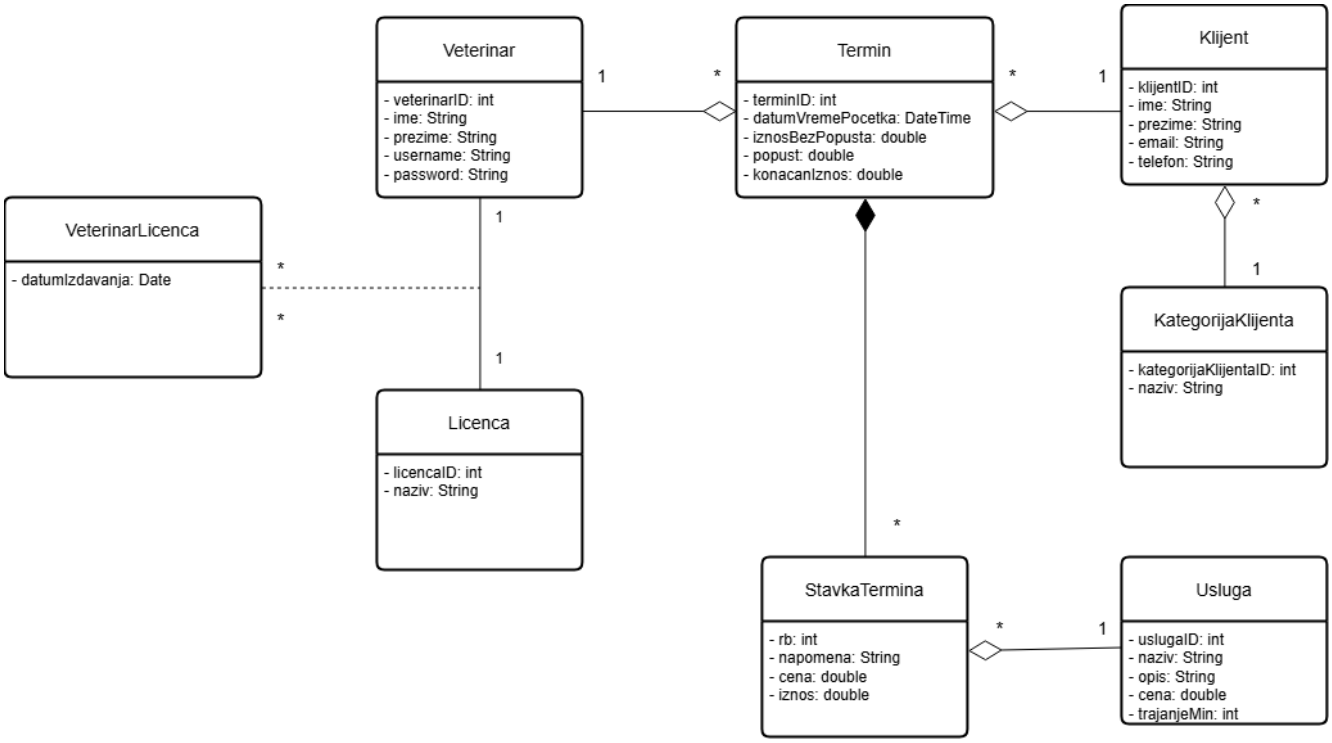
Операција:vratiListuTermin(KriterijumTermin, *List<Termin>*):signal;
Веза са СК: СК2
Предуслови:
Постуслови: *Пронађена је листа тражених објеката класе Термин.*

8. Уговор UG8: vratiListuSviKlijent

Операција: vratiListuKlijent(*List<Klijent>*):signal;
Веза са СК: СК1, СК3
Предуслови:
Постуслови: *Пронађена је листа свих објеката класе Клијент.*

3.4 Структура софтверског система – Концептуални (доменски) модел

Тип концептуалног модела 1:



Слика 1. Концептуални модел ветеринарске ординације

3.5 Структура софтверског система – Релациони модел

На основу концептуалног модела се прави релациони модел. Релациони модел добијен из **првог** типа концептуалног модела:

1. Veterinar (veterinarID, ime, prezime, username, password)
2. Usluga (uslugaID, naziv, opis, cena, trajanjeMin)
3. KategorijaKlijenta (kategorijaKlijentaID, naziv)
4. Licenca (LicencaID, naziv)
5. Klijent (klijentID, ime, prezime, email, telefon, *kategorijaKlijentaID*)
6. Termin (terminID, datumVremePocetka, iznosBezPopusta, popust, konacanIznos, *veterinarID*, *klijentID*)
7. StavkaTermina (*terminID*, rb, napomena, cena, iznos, *uslugaID*)
8. VeterinarLicenca (*veterinarID*, *licencaID*, datumIzdavanja)

3.6 Табела структурних и вредносних ограничења релационог модела

За сваку релацију се прави табела структурних и вредносних ограничења.

Табела структурних и вредносних ограничења релационог модела који је добијен из **првог** типа концептуалног модела:

1.Табела Veterinar		**Просто вредносно ограничење**		**Сложено вредносно ограничење**		**Структурно ограничење**
Атрибути	***Назив***	**Тип атрибута**	**Вредност атрибута**	**Међузав. атрибута једне табеле**	**Међузав.атрибута више табела**	INSERT / UPDATE CASCADES Termin, VeterinarLicenca DELETE RESTRICTED Termin, VeterinarLicenca
	veterinarID	Integer	Not null and >0			
	ime	String	Not null			
	prezime	String	Not null			
	username	String	Not null			
	password	String	Not null			

2.Табела Usluga		**Просто вредносно ограничење**		**Сложено вредносно ограничење**		**Структурно ограничење**
Атрибути	***Назив***	**Тип атрибута**	**Вредност атрибута**	**Међузав. атрибута једне табеле**	**Међузав.атрибута више табела**	INSERT / UPDATE CASCADES StavkaTermina DELETE RESTRICTED StavkaTermina
	uslugaID	Integer	Not null and >0			
	naziv	String	Not null			
	opis	String	Not null			
	cena	Double	Not null and >0			
	trajanjeMin	Integer	Not null and >0	trajanjeMin > 15 && trajanjeMin < 180		

3.Табела KategorijaKlijenta		Просто вредносно ограничење		Сложено вредносно ограничење		Структурно ограничење
Атрибути	***Назив***	**Тип атрибута**	**Вредност атрибута**	**Међузав. атрибута једне табеле**	**Међузав.атрибута више табела**	INSERT / UPDATE CASCADES Klijent DELETE RESTRICTED Klijent
	kategorijaKlijentaID	Integer	Not null and >0			
	naziv	String	Not null			

4.Табела Licenca		Просто вредносно ограничење		Сложено вредносно ограничење		Структурно ограничење
Атрибути	***Назив***	**Тип атрибута**	**Вредност атрибута**	**Међузав. атрибута једне табеле**	**Међузав.атрибута више табела**	INSERT / UPDATE CASCADES VeterinarLicenca DELETE RESTRICTED VeterinarLicenca
	licencaID	Integer	Not null and >0			
	naziv	String	Not null			

5.Табела Klijent		Просто вредносно ограничење		Сложено вредносно ограничење		Структурно ограничење
Атрибути	***Назив***	**Тип атрибута**	**Вредност атрибута**	**Међузав. атрибута једне табеле**	**Међузав.атрибута више табела**	INSERT RESTRICTED KategorijaKlijenta UPDATE RESTRICTED KategorijaKlijenta UPDATE CASCADES Termin DELETE RESTRICTED Termin
	klijentID	Integer	Not null and >0			
	kategorijaKlijentaID	Integer	Not null and >0			
	ime	String	Not null			
	prezime	String	Not null			
	email	String	Not null			
	telefon	String	Not null			

6. Табела Termin		Просто вредносно ограничење		Сложено вредносно ограничење		Структурно ограничење
Атрибути	***Назив***	**Тип атрибута**	**Вредност атрибута**	**Међузав. атрибута једне табеле**	**Међузав.атрибута више табела**	INSERT RESTRICTED Klijent, Veterinar UPDATE RESTRICTED Klijent, Veterinar UPDATE CASCADES Termina DELETE CASCADES StavkaTermina
	terminID	Integer	Not null and >0			
	veterinarID	Integer	Not null and >0			
	klijentID	Integer	Not null and >0			
	datumVreme Pocetka	Date	Not null			
	iznosBezPopusta	Double	Not null and >0		iznosBezPopusta= SUM(StavkaTermina.iznos)	
	popust	Double	Not null and >0			
	konacanIznos	Double	Not null and >0	konacanIznos = iznosBezPopusta * (1 – popust/100)		

7. Табела StavkaTermina		Просто вредносно ограничење		Сложено вредносно ограничење		Структурно ограничење
Атрибути	***Назив***	**Тип атрибута**	**Вредност атрибута**	**Међузав. атрибута једне табеле**	**Међузав.атрибута више табела**	INSERT RESTRICTED Termin, Usluga UPDATE RESTRICTED Termin, Usluga DELETE /
	terminID	Integer	Not null and >0			
	rb	Integer	Not null and >0			
	uslugaID	Integer	Not null and >0			
	napomena	String	Not null			
	cena	Double	Not null and >0		cena = Usluga.cena	
	iznos	Double	Not null and >0	iznos = cena		

8. Табела VeterinarLicenca		Просто вредносно ограничење		Сложено вредносно ограничење		Структурно ограничење
Атрибути	***Назив***	**Тип атрибута**	**Вредност атрибута**	**Међузав. атрибута једне табеле**	**Међузав.атрибута више табела**	INSERT RESTRICTED Veterinar, Licenca UPDATE RESTRICTED Veterinar, Licenca DELETE /
	veterinarID	Integer	Not null and >0			
	licencaID	Integer	Not null and >0			
	datumIzdavanja	Date	Not null			

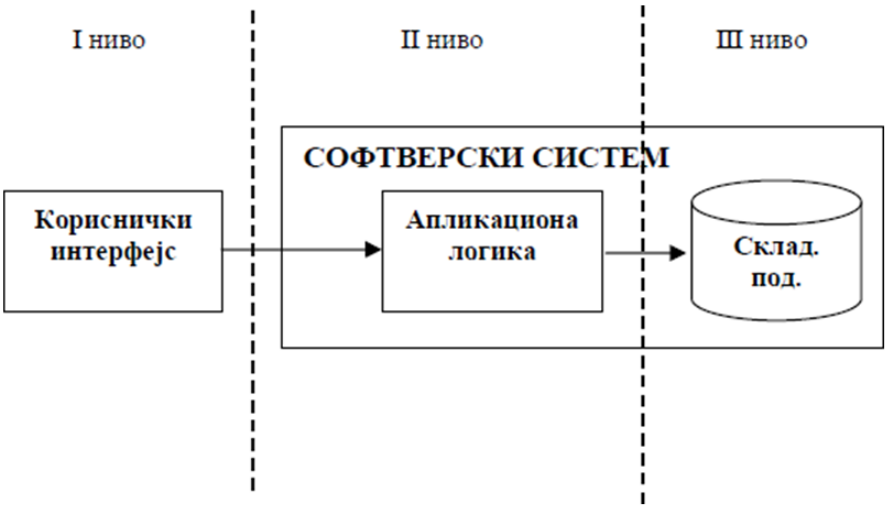
4. Пројектовање

Фаза пројектовања описује физичку структуру и понашање софтверског система. Пројектовање архитектуре софтверског система обухвата пројектовање корисничког интерфејса (пројектовање контролера корисничког интерфејса и екранских форми), апликационе логике (пројектовање контролера апликационе логике и пословне логике) и складишта података (брокер базе података).

Архитектура система је тронивојска и састоји се од следећих нивоа:

- кориснички интерфејс
- апликациона логика
- складиште података

Ниво корисничког интерфејса ја на страни клијента, док су апликациона логика и складиште на страни сервера.

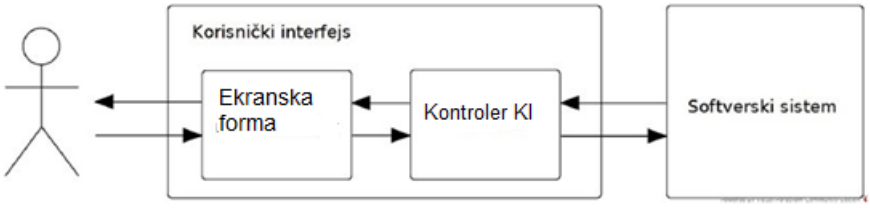


Слика 2. Тронивојска архитектура

4.1 Пројектовање корисничког интерфејса

Кориснички интерфејс представља улазно-излазну реализацију софтверског система. Састоји се од:

1. Екранске форме
2. Контролера корисничког интерфејса



Слика 3. Структура корисничког интерфејса

4.2 Пројектовање екранских форми

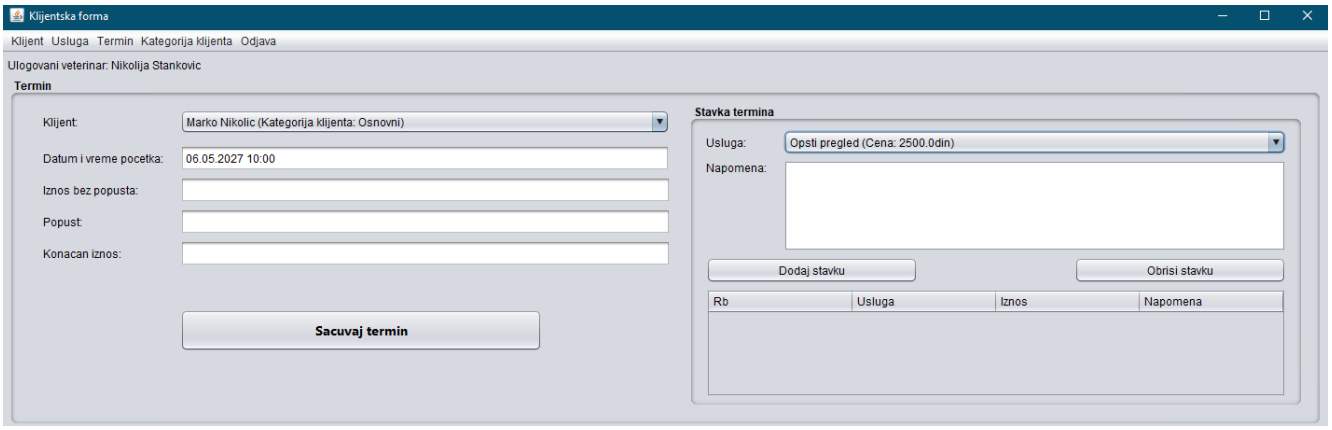
СК1- Убаци термин

Назив СК
Убаци термин

Актори СК
Ветеринар

Учесници СК
Ветеринар, кориснички интерфејс (клијентски програм) и систем (серверски програм)

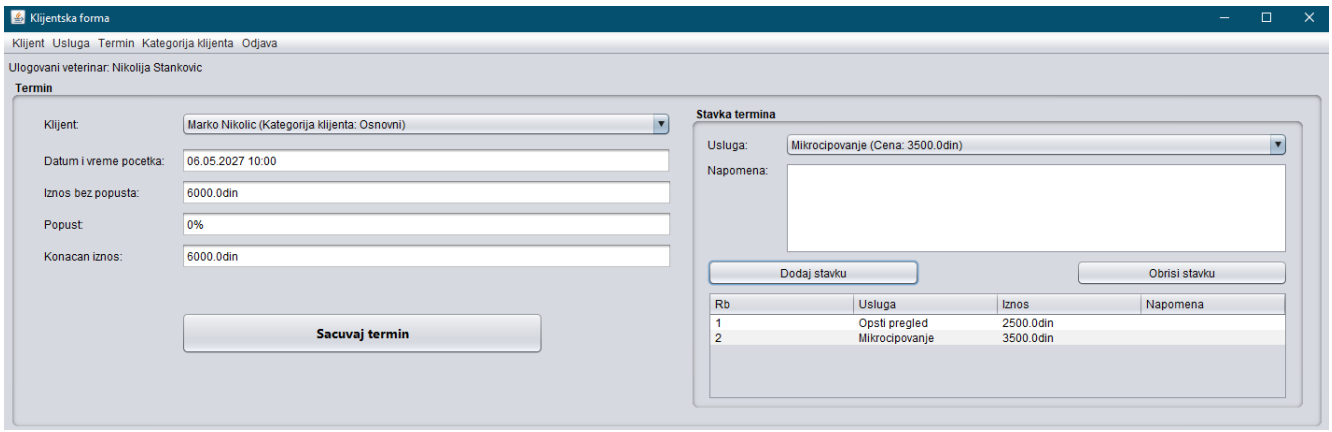
Предуслови: Кориснички интерфејс (клијентски програм) и систем (серверски програм) су покренути. Ветеринар је пријављен под својом шифром. Систем приказује форму за рад са термином. *Учитане су листе: а) Ветеринар b) Клијент c) Услуга*



Слика 4. форма за убациване новог термина

Основни сценарио СК:

1. Ветеринар **уноси** податке о термину. (АПУСО)
2. Ветеринар **контролише** да ли је коректно унео податке о термину. (АНСО)

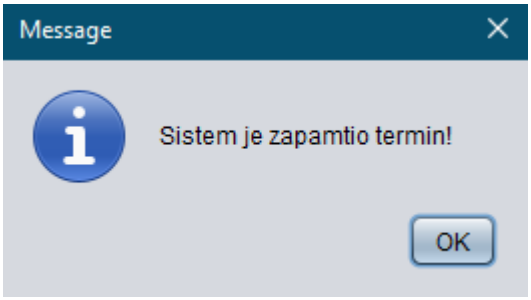


Слика 5. Унос података о термину

3. Ветеринар **позива** систем да запамти податке о термину. (АПСО)

Опис акције:
Ветеринар кликом на дугме „Sacuvaj termin“ позива системску операцију **ubaciTermin(Termin)**

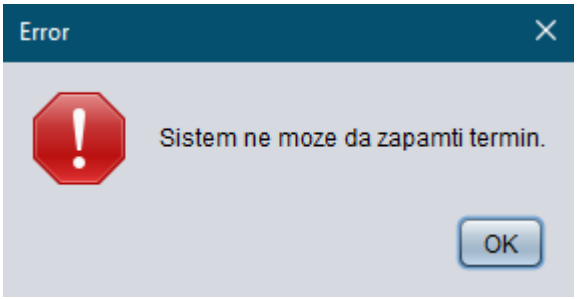
4. Систем **памти** податке о термину. (СО)
5. Систем **приказује** ветеринару поруку: “Систем је запамтио термин.” (ИА)



Слика 6. Успешно убацивање термина

Алтернативна сценарија:

5.1 Уколико систем не може да запамти податке о термину он **приказује** ветеринару поруку: “Систем не може да запамти термин”. (ИА)



Слика 7. Грешка при убацивању термина

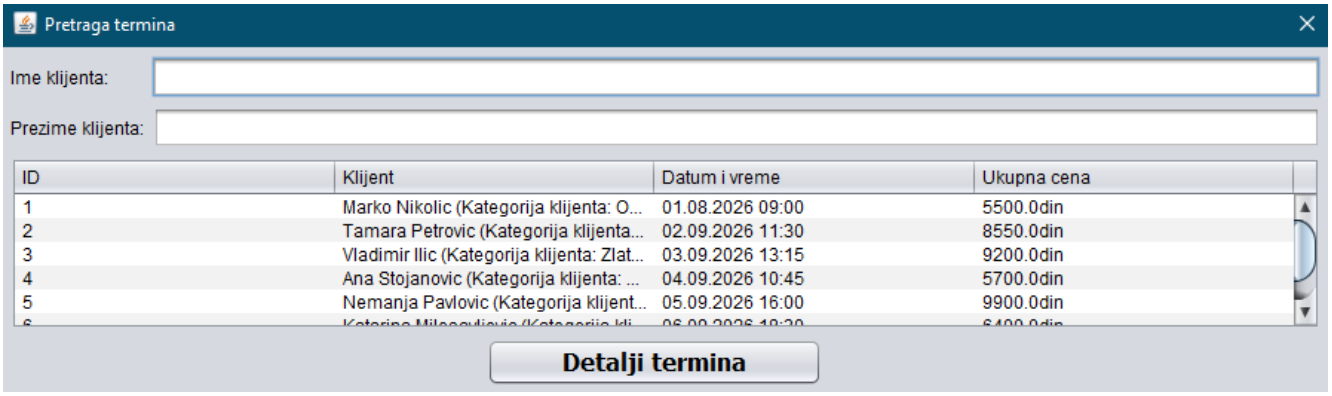
СК2- Претражи термин

Назив СК
Претражи термин

Актори СК
Ветеринар

Учесници СК
Ветеринар, кориснички интерфејс (клијентски програм) и систем (серверски програм)

Предуслови: Кориснички интерфејс (клијентски програм) и систем (серверски програм) су покренути. Ветеринар је пријављен под својом шифром. Кориснички интерфејс приказује форму за рад са термином. На наведеној екранској форми су дефинисани *критеријуми,* који се односе на: a) Термин b) Ветеринар c) Клијент d) Услуга, који ће да врате листу термина.



Слика 8. Форма за претрагу рачуна

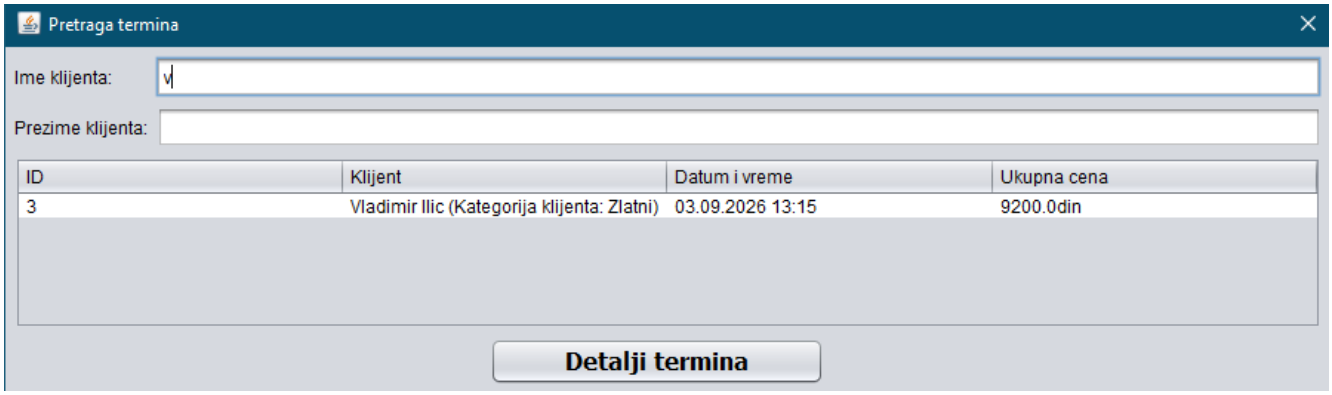
Основни сценарио СК:

1. Ветеринар **бира** критеријуме на основу којих претражује термине. (АПУСО)
2. Ветеринар **позива** систем да нађе термине по задатим критеријумима. (АПСО)

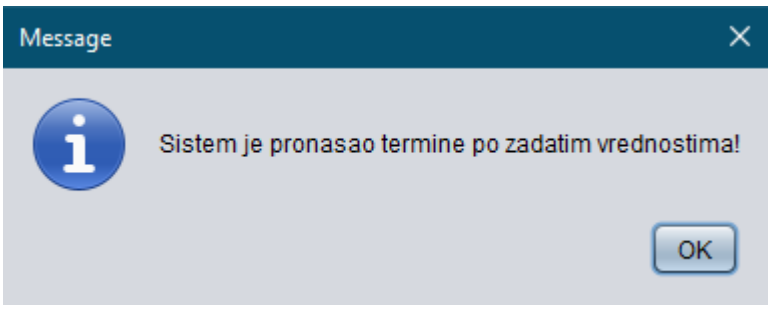
Опис акције:

При самом отварању форме за претрагу термина, иницијално се позива системска операција **vratiListuSviTermin(),** а учитани термини се приказују у табели. Ветеринар у поља претраге уноси критеријум по коме претражује термине, при чему се позива системска операција **pretraziTermin(ZadataVrednost, List<Termin>)**

3. Систем **тражи** термине по задатим критеријумима. (СО)
4. Систем **приказује** ветеринару термине и поруку: "Систем је нашао термине по задатим критеријумима". (ИА)



Слика 9. Успешна претрага термина



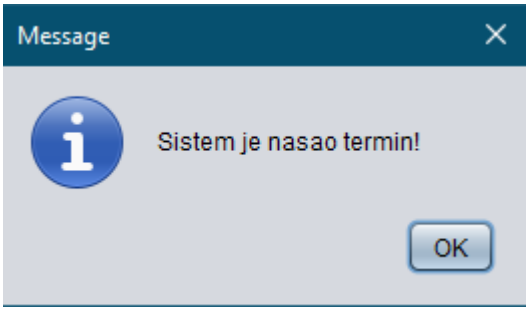
Слика 10. Систем приказује поруку ветеринару

5. Ветеринар **бира** термин. (АПУСО)
6. Ветеринар **позива** систем да нађе термин. (АПСО)

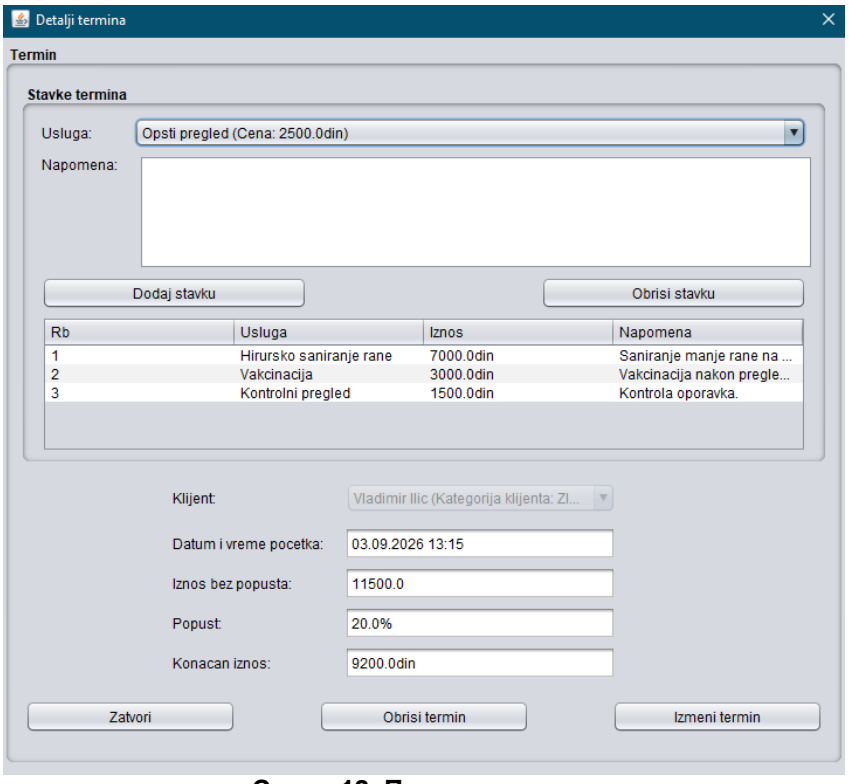
Опис акције:

Ветеринар кликом на дугме „Detalji termina“ позива системску операцију **pretraziTermin(Termin)**

7. Систем **тражи** термин. (СО)
8. Систем **приказује** ветеринару термин и поруку: “Систем је нашао термин”. (ИА)



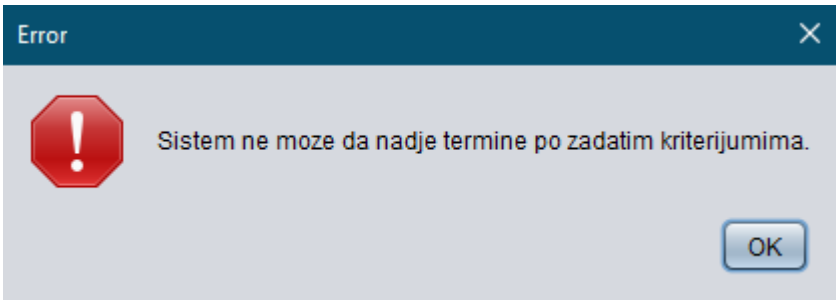
Слика 11. Систем приказује поруку ветеринару



Слика 12. Приказ термина

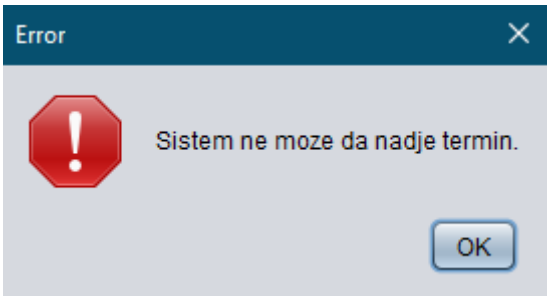
Алтернативна сценарија:

4.2 Уколико систем не може да нађе термине он **приказује** ветеринару поруку: “Систем не може да нађе термине по задатим критеријумима”. Прекида се извршење сценарија. (ИА)



Слика 13. Грешка при претрази термина

8.1 Уколико систем не може да нађе термин он **приказује** ветеринару поруку: “Систем не може да нађе термин”. (ИА)



Слика 14. Грешка при претрази термина

СК3- Промени термин

Назив СК
Промени термин

Актори СК
Ветеринар

Учесници СК
Ветеринар, кориснички интерфејс (клијентски програм) и систем (серверски програм)

Предуслови: Кориснички интерфејс (клијентски програм) и систем (серверски програм) су покренути. Ветеринар је пријављен под својом шифром. Кориснички интерфејс приказује форму за рад са термином. На наведеној екранској форми су дефинисани *критеријуми,* који се односе на: a) Термин b) Ветеринар c) Клијент d) Услуга, који ће да врате листу термина. Учитане су листе: a) Ветеринар b) Клијент c) Услуга

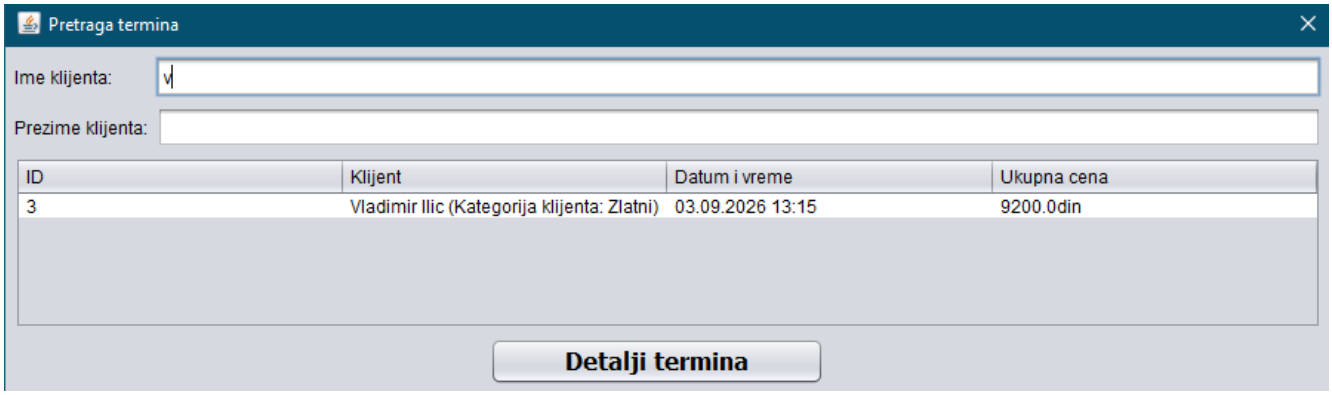
Основни сценарио СК:

1. Ветеринар **бира** критеријуме на основу којих претражује термине. (АПУСО)
2. Ветеринар **позива** систем да нађе термине по задатим критеријумима. (АПСО)

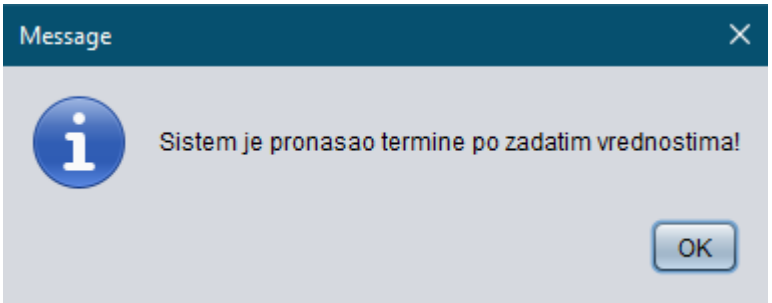
Опис акције:

При самом отварању форме за претрагу термина, иницијално се позива системска операција **vratiListuSviTermin(),** а учитани термини се приказују у табели. Ветеринар у поља претраге уноси критеријум по коме претражује термине, при чему се позива системска операција **pretraziTermin(ZadataVrednost, List<Termin>)**

3. Систем **тражи** термине по задатим критеријумима. (СО)
4. Систем **приказује** ветеринару термине и поруку: "Систем је нашао термине по задатим критеријумима". (ИА)



Слика 15. Успешна претрага термина



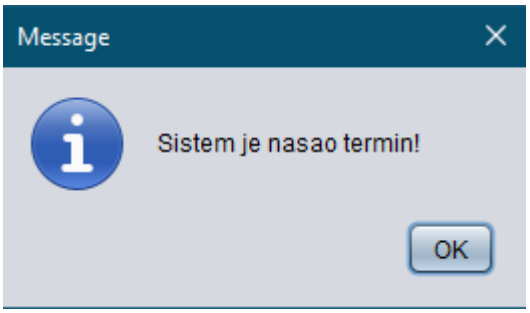
Слика 16. Систем приказује поруку ветеринару

5. Ветеринар **бира** термин. (АПУСО)
6. Ветеринар **позива** систем да нађе термин. (АПСО)

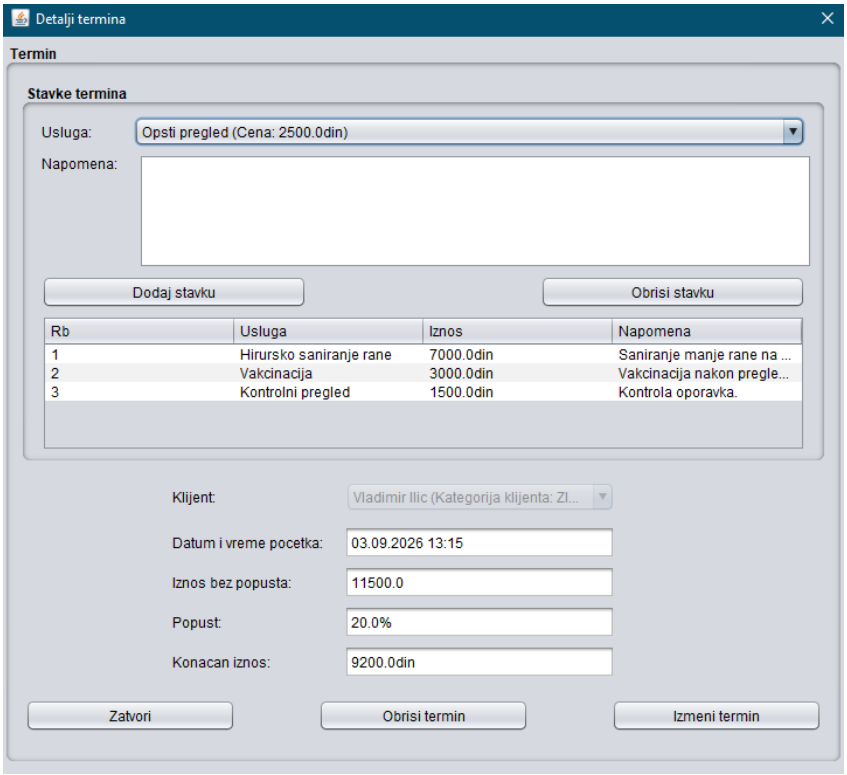
Опис акције:

Ветеринар кликом на дугме „Detalji termina“ позива системску операцију **pretraziTermin(Termin)**

7. Систем **тражи** термин. (СО)
8. Систем **приказује** ветеринару термин и поруку: “Систем је нашао термин”. (ИА)



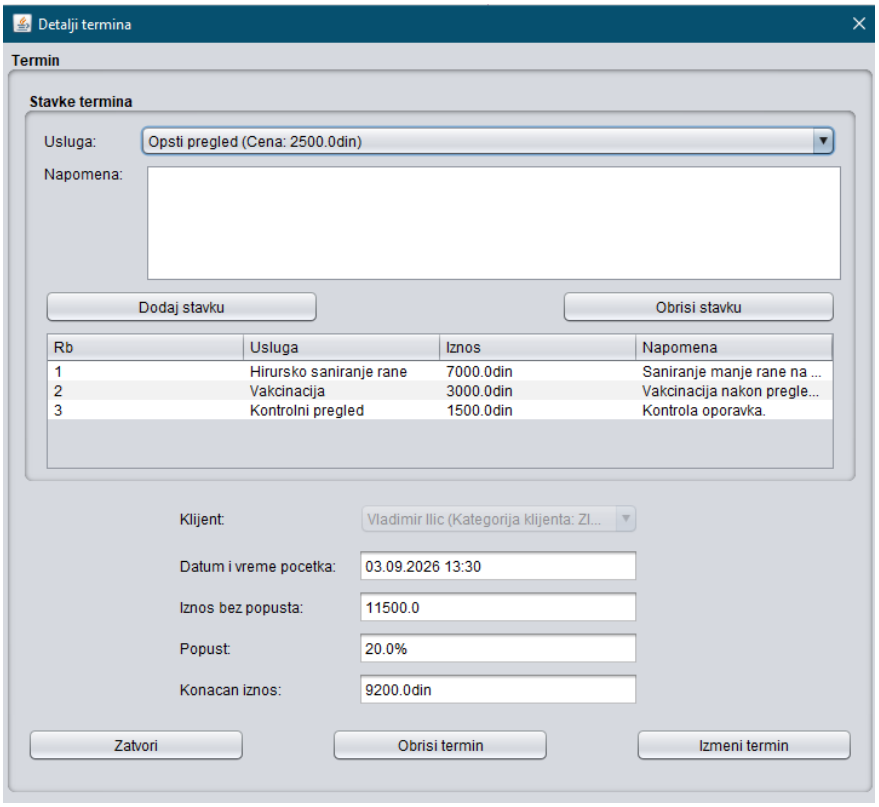
Слика 17. Систем приказује поруку ветеринару



Слика 18. Приказ термина

9. Ветеринар **уноси (мења)** податке о термину. (АПУСО)

10. Ветеринар **контролише** да ли је коректно унео податке о термину. (АНСО)

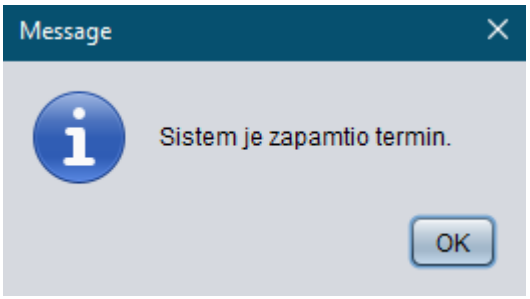


Слика 19. Ветеринар контролише да ли је коректно унео податке о термину

11. Ветеринар **позива** систем да запамти податке о термину. (АПСО)

Опис акције:
Ветеринар кликом на дугме „Izmeni termin“ позива системску операцију **promeniTermin(Termin)**

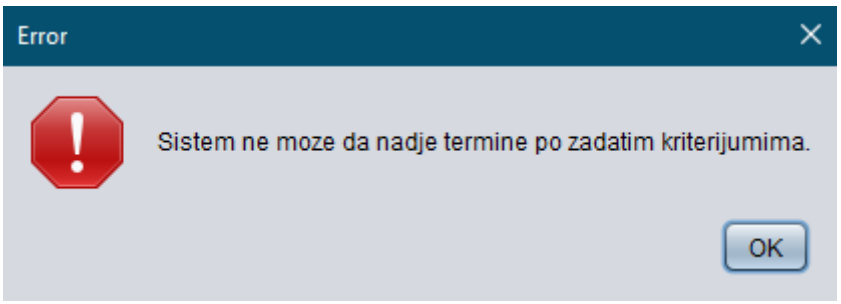
12. Систем **памти** податке о термину. (СО)
13. Систем **приказује** ветеринару поруку: “Систем је запамтио термин.” (ИА)



Слика 20. Успешна измена термина

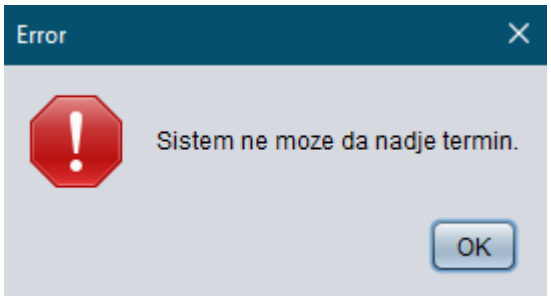
Алтернативна сценарија:

4.1 Уколико систем не може да нађе термине он **приказује** ветеринару поруку: “Систем не може да нађе термине по задатим критеријумима”. Прекида се извршење сценарија. (ИА)



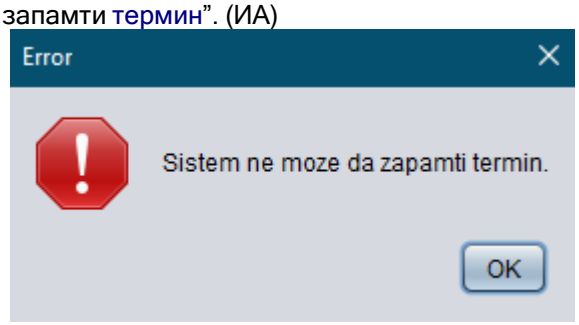
Слика 21. Грешка при претрази термина

8.2 Уколико систем не може да нађе термин он **приказује** ветеринару поруку: “Систем не може да нађе термин”. Прекида се извршење сценариа. (ИА)



Слика 22. Грешка при претрази термина

13.2 Уколико систем не може да запамти податке о термину он **приказује** ветеринару поруку: “Систем не може да запамти термин”. (ИА)



Слика 23. Грешка при измени термина

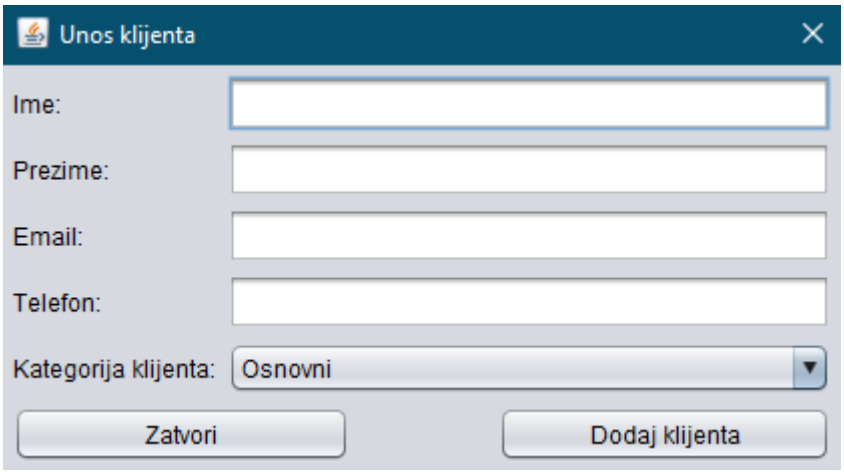
СК4- Убаци клијент

Назив СК
Убаци клијент

Актори СК
Ветеринар

Учесници СК
Ветеринар, кориснички интерфејс (клијентски програм) и систем (серверски програм)

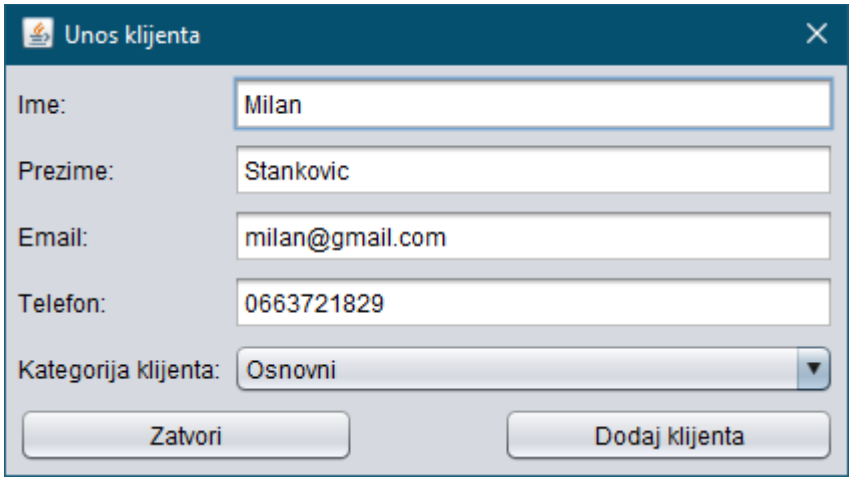
Предуслови: Кориснички интерфејс (клијентски програм) и систем (серверски програм) су покренути. Ветеринар је пријављен под својом шифром. Систем приказује форму за рад са клијентом. *Учитане су листе: а) Категорија клијента*



Слика 24. Форма за унос клијента

Основни сценарио СК:

1. Ветеринар **уноси** податке о клијенту. (АПУСО)
2. Ветеринар **контролише** да ли је коректно унео податке о клијенту. (АНСО)

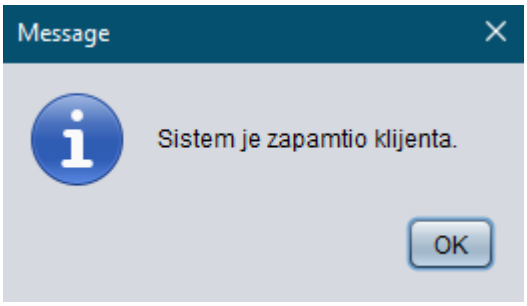


Слика 25. Ветеринар уноси клијента

3. Ветеринар **позива** систем да запамти податке о клијенту. (АПСО)

Опис акције:
Ветеринар кликом на дугме „Dodaj klijenta“ позива системску операцију **ubaciKlijent(Klijent)**

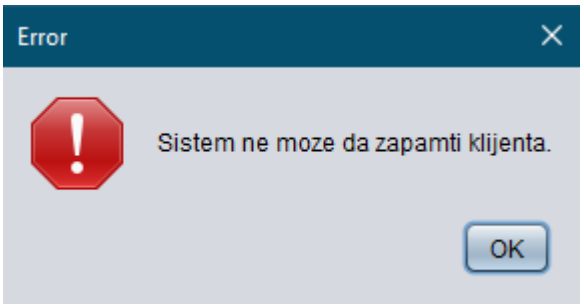
4. Систем **памти** податке о клијенту. (СО)
5. Систем **приказује** ветеринару поруку: “Систем је запамтио клијента.” (ИА)



Слика 26. Успешно убацивање клијента

Алтернативна сценарија:

5.1 Уколико систем не може да запамти податке о клијенту он **приказује** ветеринару поруку: “Систем не може да запамти клијента”. (ИА)



Слика 27. Грешка при убацивању клијента

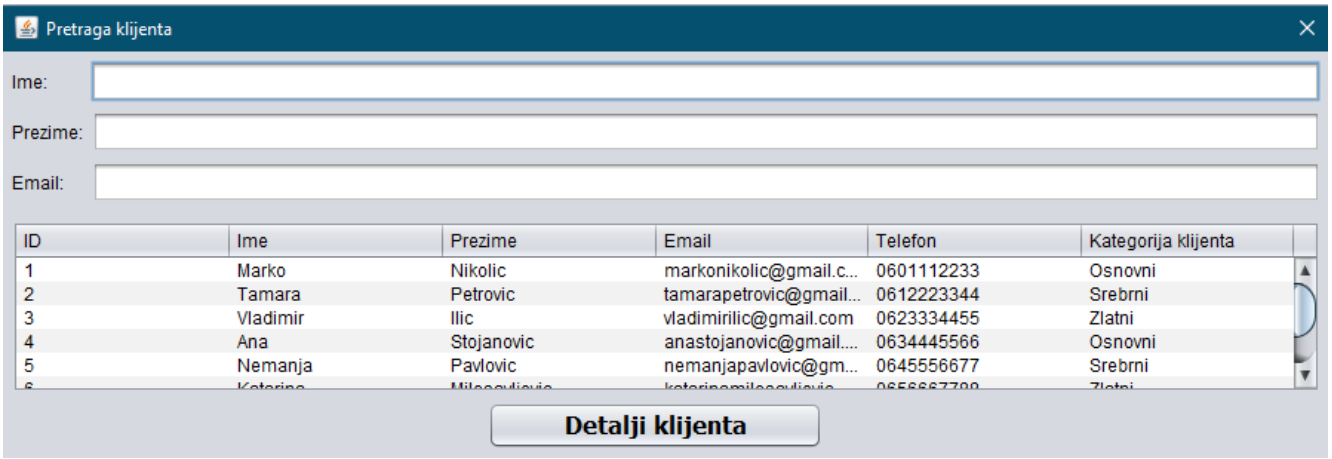
СК5- Претражи клијент

Назив СК
Претражи клијент

Актори СК
Ветеринар

Учесници СК
Ветеринар, кориснички интерфејс (клијентски програм) и систем (серверски програм)

Предуслови: Кориснички интерфејс (клијентски програм) и систем (серверски програм) су покренути. Ветеринар је пријављен под својом шифром. Кориснички интерфејс приказује форму за рад са клијентом. На наведеној екранској форми су дефинисани *критеријуми,* који се односе на: a) Клијент b) Категорија клијента, који ће да врате листу клијената.



Слика 28. Форма за претрагу клијента

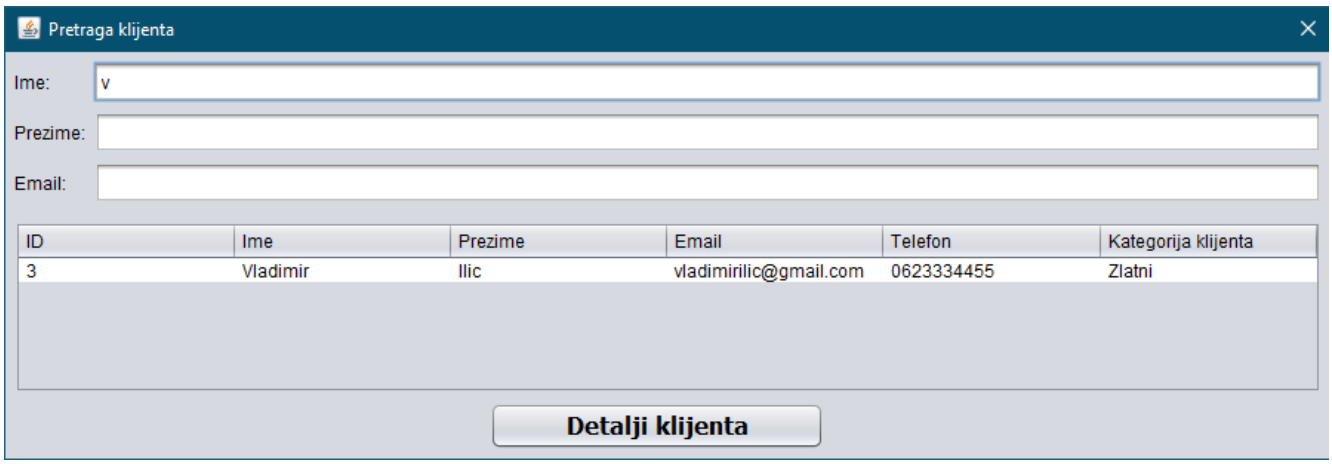
Основни сценарио СК:

1. Ветеринар **бира** критеријуме на основу којих претражује клијенте. (АПУСО)
2. Ветеринар **позива** систем да нађе клијенте по задатим критеријумима. (АПСО)

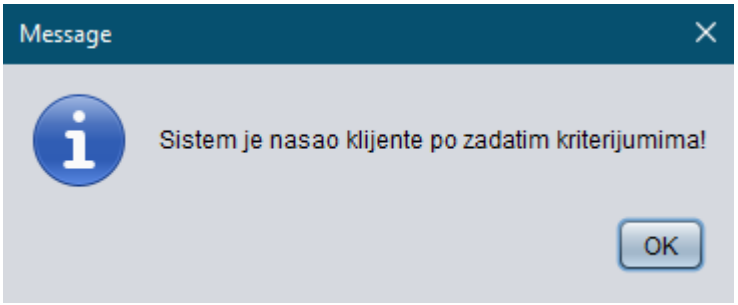
Опис акције:

При самом отварању форме за претрагу клијената, иницијално се позива системска операција **vratiListuSviKlijent(),** а учитани клијенти се приказују у табели. Ветеринар у поља претраге уноси критеријум по коме претражује клијенте, при чему се позива системска операција **pretraziKlijent(ZadataVrednost, List<Klijent>)**

3. Систем **тражи** клијенте по задатим критеријумима. (СО)
4. Систем **приказује** ветеринару клијенте и поруку: "Систем је нашао клијенте по задатим критеријумима". (ИА)



Слика 29. Успешна претрага клијента



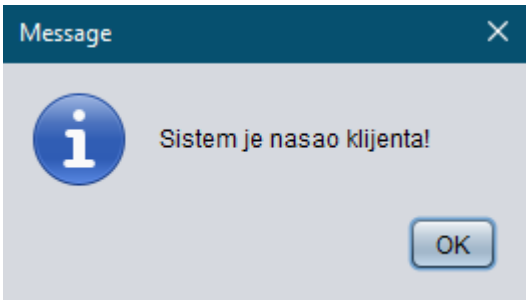
Слика 30. Систем приказује поруку ветеринару

5. Ветеринар **бира** клијента. (АПУСО)
6. Ветеринар **позива** систем да нађе клијента. (АПСО)

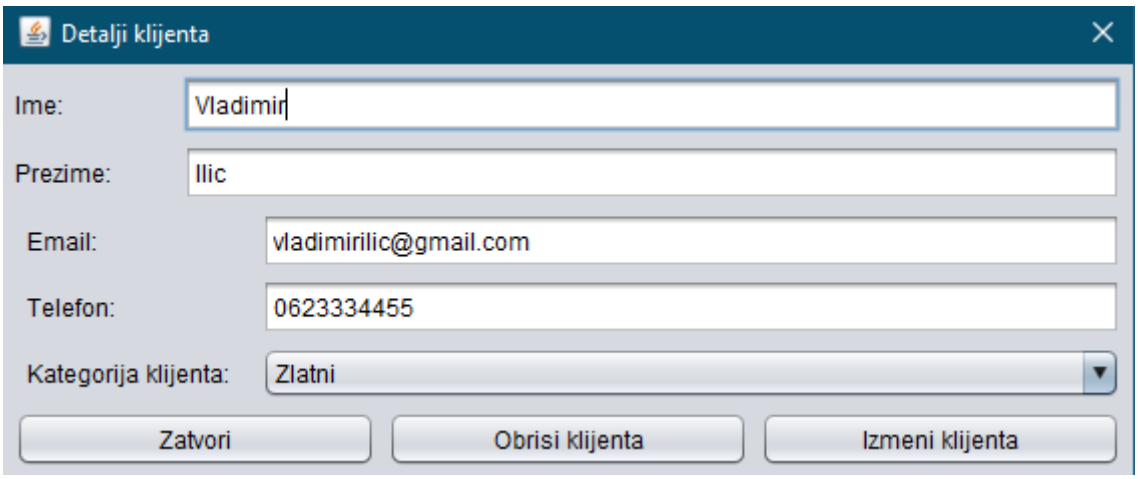
Опис акције:

Ветеринар кликом на дугме „Detalji klijenta“ позива системску операцију **pretraziKlijent(Klijent)**

7. Систем **тражи** клијента. (СО)
8. Систем **приказује** ветеринару клијента и поруку: “Систем је нашао клијента”. (ИА)



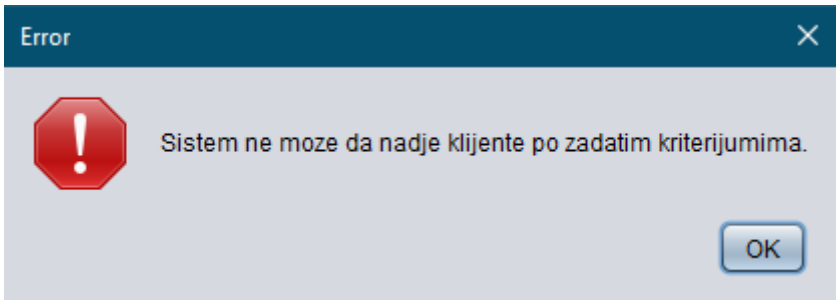
Слика 31. Систем приказује поруку ветеринару



Слика 32. Приказ детаља клијента

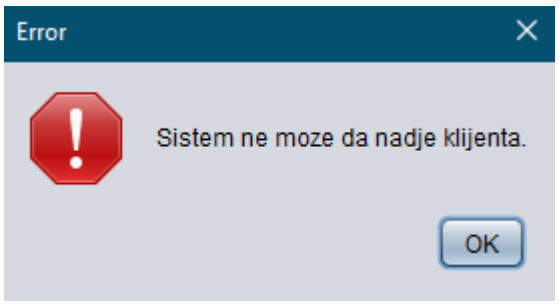
Алтернативна сценарија:

4.2 Уколико систем не може да нађе клијенте он **приказује** ветеринару поруку: “Систем не може да нађе клијенте по задатим критеријумима”. Прекида се извршење сценарија. (ИА)



Слика 33. Грешка при претрази клијента

8.1 Уколико систем не може да нађе клијента он **приказује** ветеринару поруку: “Систем не може да нађе клијента”. (ИА)



Слика 34. Грешка при претрази клијента

СК6- Промени клијент

Назив СК
Промени клијент

Актори СК
Ветеринар

Учесници СК
Ветеринар, кориснички интерфејс (клијентски програм) и систем (серверски програм)

Предуслови: Кориснички интерфејс (клијентски програм) и систем (серверски програм) су покренути. Ветеринар је пријављен под својом шифром. Кориснички интерфејс приказује форму за рад са клијентом. На наведеној екранској форми су дефинисани *критеријуми,* који се односе на: а) Клијент b) Категорија клијента, који ће да врате листу клијената. Учитане су листе: а) Категорија клијента

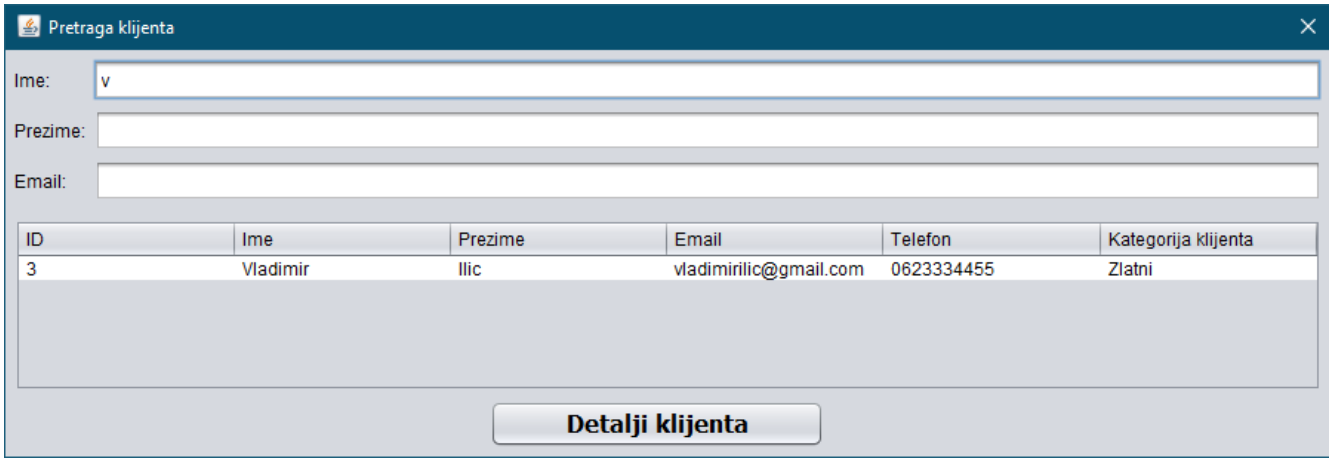
Основни сценарио СК:

1. Ветеринар **бира** критеријуме на основу којих претражује клијенте. (АПУСО)
2. Ветеринар **позива** систем да нађе клијенте по задатим критеријумима. (АПСО)

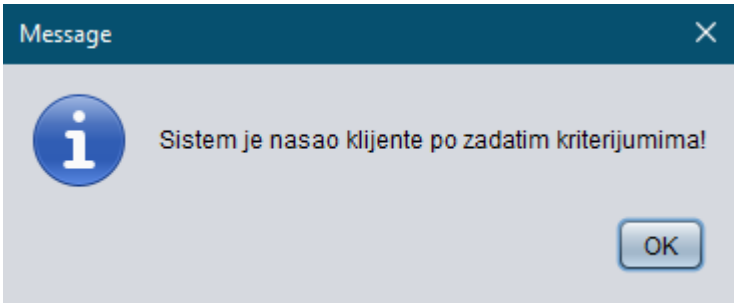
Опис акције:

При самом отварању форме за претрагу клијената, иницијално се позива системска операција **vratiListuSviKlijent(),** а учитани клијенти се приказују у табели. Ветеринар у поља претраге уноси критеријум по коме претражује клијенте, при чему се позива системска операција **pretraziKlijent(ZadataVrednost, List<Klijent>)**

3. Систем **тражи** клијенте по задатим критеријумима. (СО)
4. Систем **приказује** ветеринару клијенте и поруку: "Систем је нашао клијенте по задатим критеријумима". (ИА)



Слика 35. Успешна претрага клијента



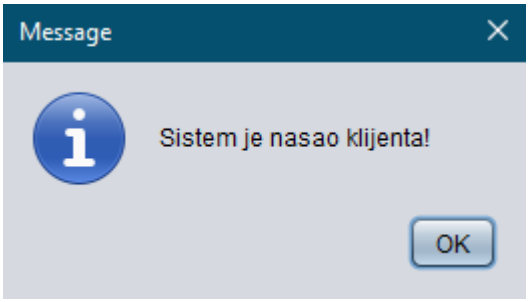
Слика 36. Систем приказује поруку ветеринару

5. Ветеринар **бира** клијента. (АПУСО)
6. Ветеринар **позива** систем да нађе клијента. (АПСО)

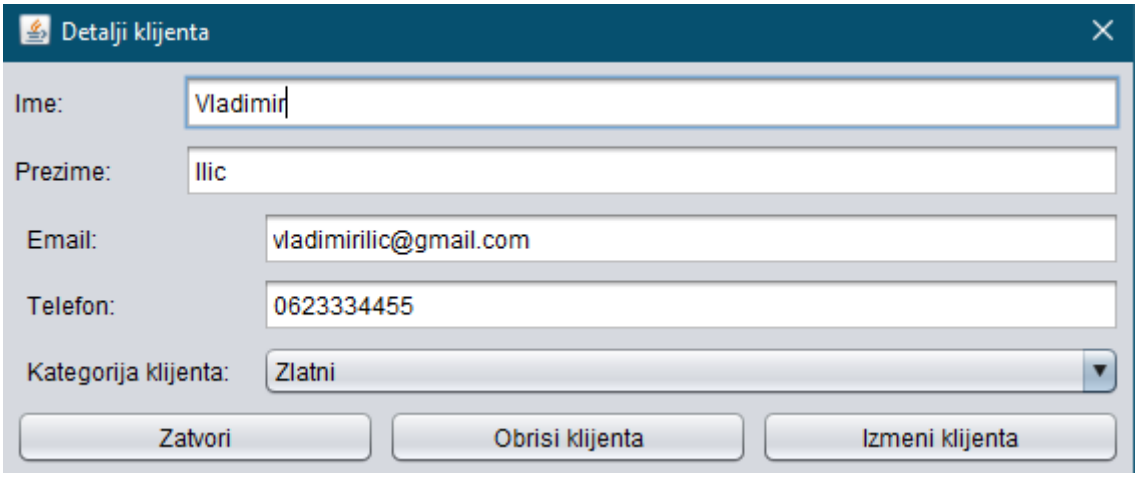
Опис акције:

Ветеринар кликом на дугме „Detalji klijenta“ позива системску операцију **pretraziKlijent(Klijent)**

7. Систем **тражи** клијента. (СО)
8. Систем **приказује** ветеринару клијента и поруку: “Систем је нашао клијента”. (ИА)

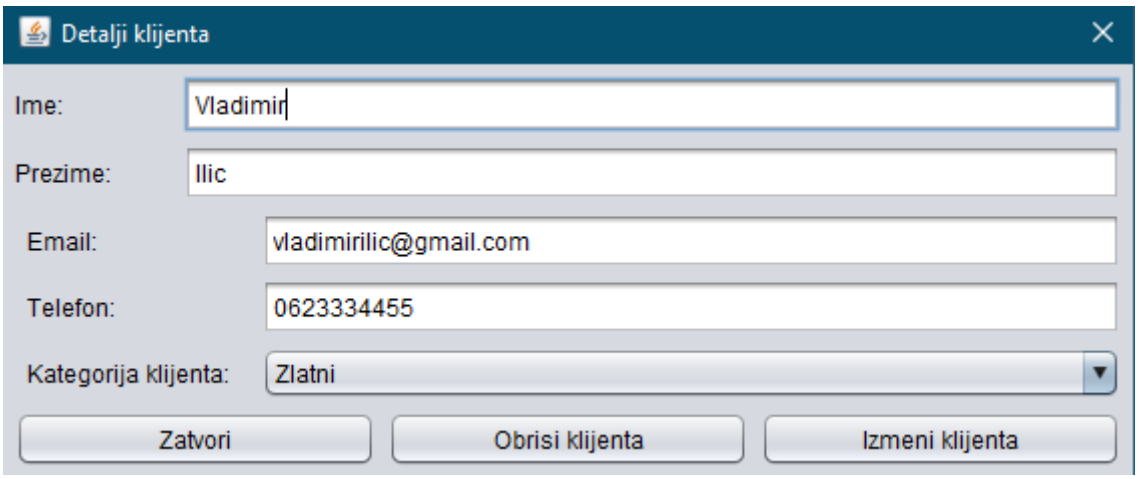


Слика 37. Систем приказује поруку ветеринару



Слика 38. Приказ детаља клијента

9. Ветеринар **уноси (мења)** податке о клијенту. (АПУСО)
10. Ветеринар **контролише** да ли је коректно унео податке о клијенту. (АНСО)

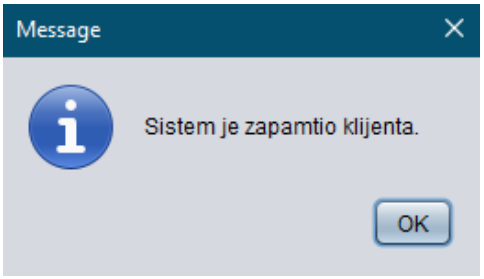


Слика 39. Ветеринар контролише да ли је коректно унео податке о клијенту

11. Ветеринар **позива** систем да запамти податке о клијенту. (АПСО)

Опис акције:
Ветеринар кликом на дугме „Izmeni klijenta“ позива системску операцију **promeniKlijent(Klijent)**

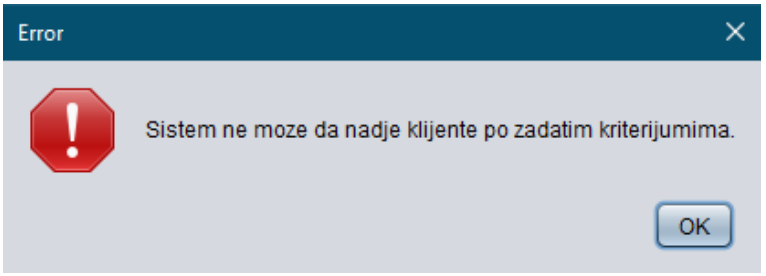
12. Систем **памти** податке о клијенту. (СО)
13. Систем **приказује** ветеринару поруку: “Систем је запамтио клијента.” (ИА)



Слика 40. Успешна измена клијента

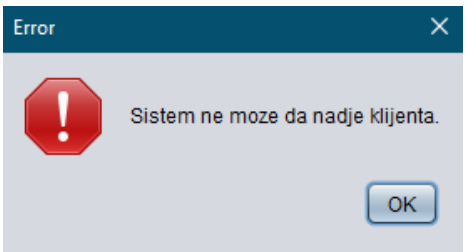
Алтернативна сценарија:

4.1 Уколико систем не може да нађе клијенте он **приказује** ветеринару поруку: “Систем не може да нађе клијенте по задатим критеријумима”. Прекида се извршење сценарија. (ИА)



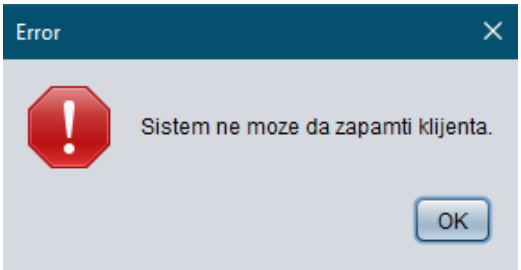
Слика 41. Грешка при претрази клијента

8.2 Уколико систем не може да нађе клијента он **приказује** ветеринару поруку: “Систем не може да нађе клијента”. Прекида се извршење сценариа. (ИА)



Слика 42. Грешка при претрази клијента

13.2 Уколико систем не може да запамти податке о клијенту он **приказује** ветеринару поруку: “Систем не може да запамти клијента”. (ИА)



Слика 43. Грешка при измени клијента

СК7- Обриши клијент

Назив СК
Обриши клијент

Актори СК
Ветеринар

Учесници СК
Ветеринар, кориснички интерфејс (клијентски програм) и систем (серверски програм)

Предуслови: Кориснички интерфејс (клијентски програм) и систем (серверски програм) су покренути. Ветеринар је пријављен под својом шифром. Кориснички интерфејс приказује форму за рад са клијентом. На наведеној екранској форми су дефинисани *критеријуми,* који се односе на: a) Клијент b) Категорија клијента, који ће да врате листу клијената.

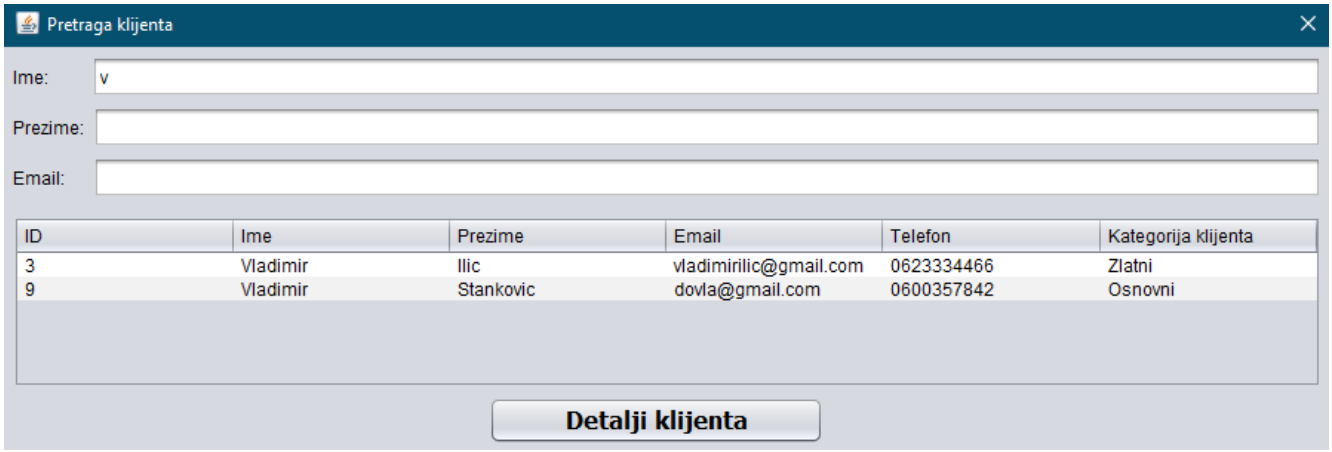
Основни сценарио СК:

1. Ветеринар **бира** критеријуме на основу којих претражује клијенте. (АПУСО)
2. Ветеринар **позива** систем да нађе клијенте по задатим критеријумима. (АПСО)

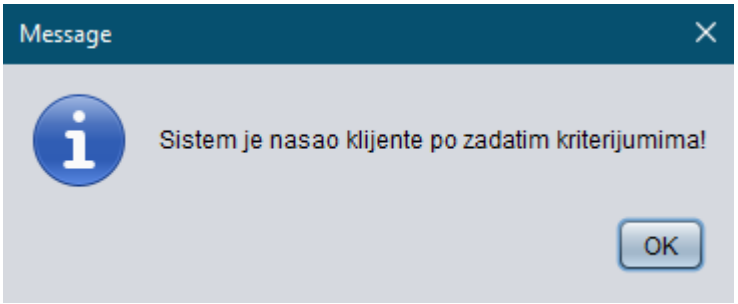
Опис акције:

При самом отварању форме за претрагу клијената, иницијално се позива системска операција **vratiListuSviKlijent(),** а учитани клијенти се приказују у табели. Ветеринар у поља претраге уноси критеријум по коме претражује клијенте, при чему се позива системска операција **pretraziKlijent(ZadataVrednost, List<Klijent>)**

3. Систем **тражи** клијенте по задатим критеријумима. (СО)
4. Систем **приказује** ветеринару клијенте и поруку: "Систем је нашао клијенте по задатим критеријумима". (ИА)



Слика 44. Успешна претрага клијента



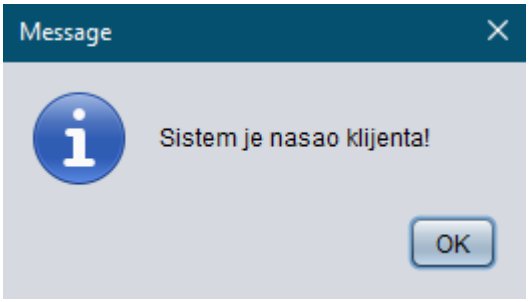
Слика 45. Систем приказује поруку ветеринару

5. Ветеринар **бира** клијента. (АПУСО)
6. Ветеринар **позива** систем да нађе клијента. (АПСО)

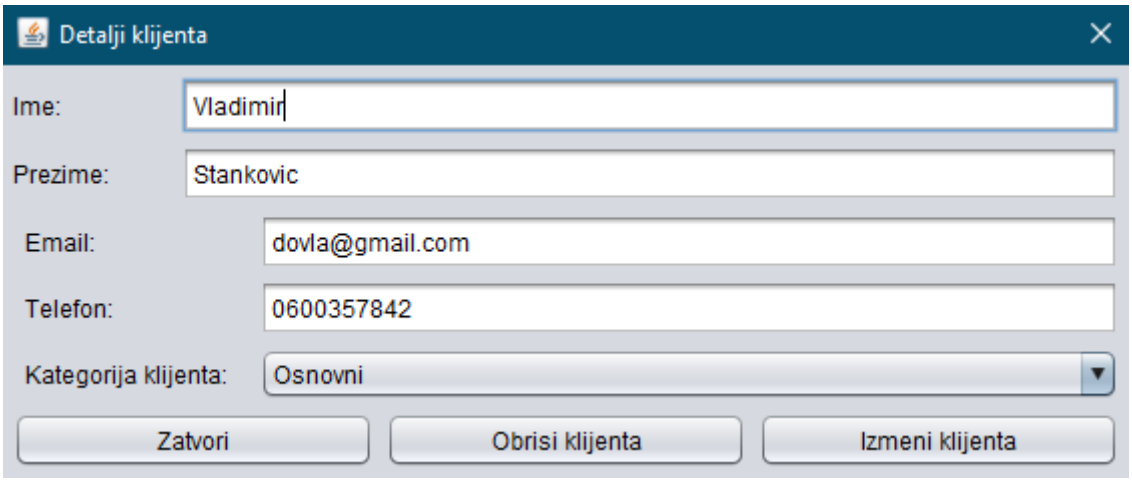
Опис акције:

Ветеринар кликом на дугме „Detalji klijenta“ позива системску операцију **pretraziKlijent(Klijent)**

7. Систем **тражи** клијента. (СО)
8. Систем **приказује** ветеринару клијента и поруку: “Систем је нашао клијента”. (ИА)



Слика 46. Систем приказује поруку ветеринару



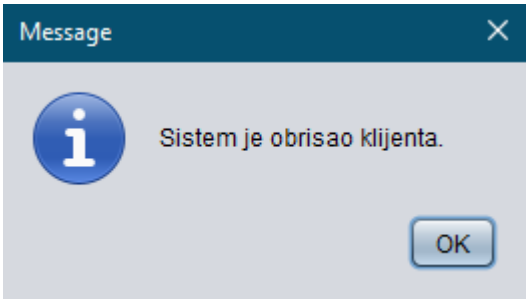
Слика 47. Приказ детаља клијента

9. Ветеринар **позива** систем да обрише клијента. (АПСО)

Опис акције:

Ветеринар кликом на дугме „Obrisi klijenta“ позива системску операцију **obrisiKlijent(Klijent)**

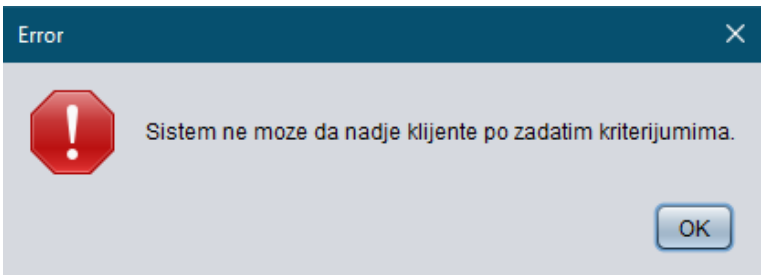
10. Систем **брише** клијента. (СО)
11. Систем **приказује** ветеринару поруку: “Систем је обрисао клијента.” (ИА)



Слика 48. Успешно брисање клијента

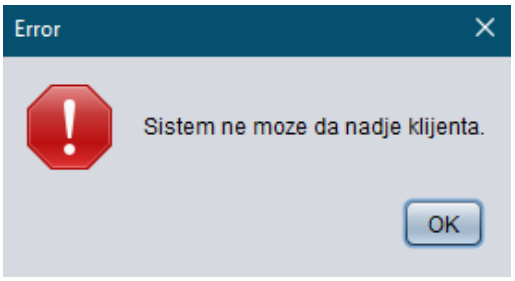
Алтернативна сценарија:

4.1 Уколико систем не може да нађе клијенте он **приказује** ветеринару поруку: “Систем не може да нађе клијенте по задатим критеријумима”. Прекида се извршење сценарија. (ИА)



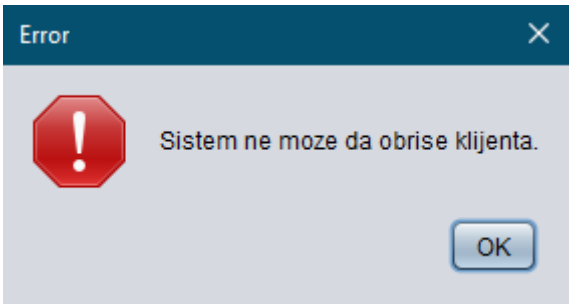
Слика 49. Грешка при претази клијента

8.2 Уколико систем не може да нађе клијента он **приказује** ветеринару поруку: “Систем не може да нађе клијента”. Прекида се извршење сценарија. (ИА)



Слика 50. Грешка при претрази клијента

11.2 Уколико систем не може да обрише клијента он **приказује** ветеринару поруку: “Систем не може да обрише клијента”. (ИА)



Слика 51. Грешка при брисању клијента

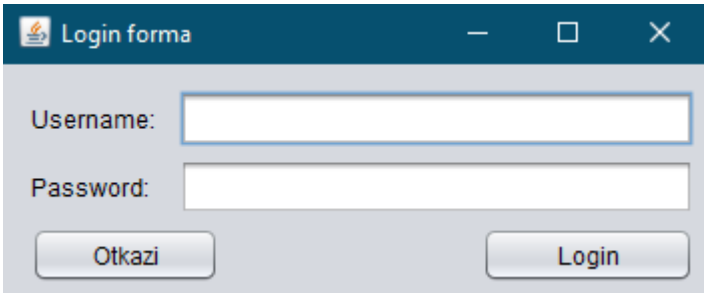
СК8- Пријави ветеринар

Назив СК
Пријави ветеринар

Актори СК
Ветеринар

Учесници СК
Ветеринар, кориснички интерфејс (клијентски програм) и систем (серверски програм)

Предуслови: Кориснички интерфејс (клијентски програм) и систем (серверски програм) су покренути. Кориснички интерфејс приказује форму за ***пријављивање***.

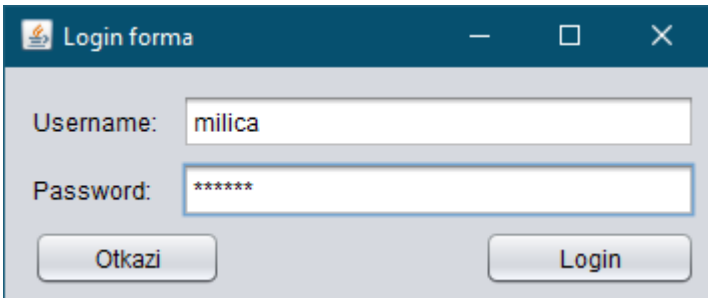


Слика 52. Форма за пријављивање

Основни сценарио СК:

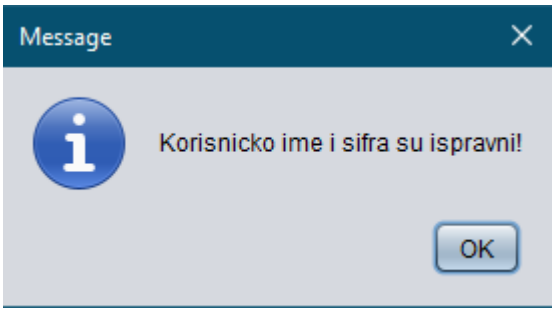
1. Ветеринар **уноси** корисничко име и шифру. (АПУСО)
2. Ветеринар **контролише** да ли је коректно унео корисничко име и шифру. (АНСО)
3. Ветеринар **позива** систем да провери корисничко име и шифру. (АПСО)

Опис акције:
Ветеринар кликом на дугме „login“ позива системску операцију **login(Veterinar)**



Слика 53. Ветеринар позива систем да провери унете креденцијале

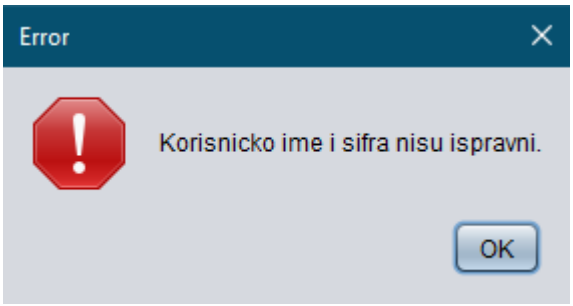
4. Систем **проверава** корисничко име и шифру. (СО)
5. Систем **приказује** ветеринару поруку: “*Корисничко име и шифра су исправни.*” (ИА)
6. Кориснички интерфејс **позива** главну форму и мени. (КИПГФМ)



Слика 54. Успешно пријављивање ветеринара на систем

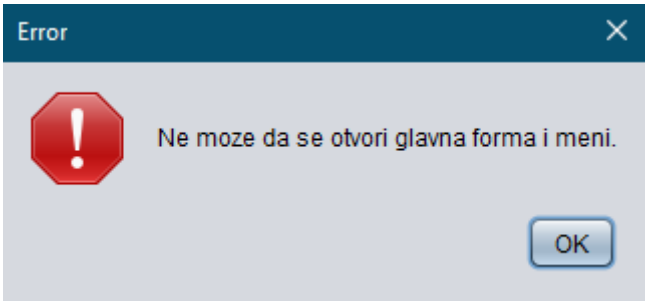
Алтернативна сценарија:

5.2 Уколико систем провером установи да корисничко име и/или шифра нису исправни он **приказује** ветеринару поруку: “*Корисничко име и шифра нису исправни”.* (ИА)



Слика 55. Грешка при пријављивању ветеринара на систем

6.1 Уколико кориснички интерфејс не може да отвори главну форму и мени **приказује** ветеринару поруку: “*Не може да се отвори главна форма и мени*”. (НПГФМ)



Слика 56. Грешка при пријављивању ветеринара на систем

4.3 Пројектовање контролера корисничког интерфејса

Контролер корисничког интерфејса је одговоран да:

1. прихвати податке које шаље екранска форма
2. конвертује податке (који се налазе у графичким елементима) у објекат који представља улазни аргумент СО која ће бити позвана
3. шаље захтев за извршење системске операције до апликационог сервера (софтверског система)
4. прихвата објекат (излаз) софтверског система настао као резултат извршења системске операције
5. конвертује објекат у податке графичких елемената

4.4 Пројектовање апликационе логике

Апликациона логика описује структуру и понашање система. Апликациони сервер се састоји из:

1. **Контролера апликационе логике** – треба да подигне серверски сокет који ће да ослушкује мрежу. Служи за комуникацију са клијентом и одговоран је да прихвати захтев за извршење системске операције од клијента и проследи га до пословне логике која је одговорна за извршење СО
2. **Пословна логика** – описана је структуром (доменске класе) и понашањем (системске операције)
3. **Брокер базе података** – служи за комуникацију између пословне логике и базе података

4.4.1 Контролер апликационе логике

Део за комуникацију подиже серверски сокет који ослушкује мрежу. Када клијентски сокет успостави конекцију са серверским сокетом, тада сервер генерише нит која ће успоставити двосмерну комуникацију са клијентом.

Софтверски систем реализован је као клијент-сервер апликација. На серверској страни је нит *ThreadServer* која садржи објекат класе *ServerSocket*. Нит константно позива методу *accept* која чека да се покрене клијентска апликација која, кад се то деси, ће покушати да се повеже на сервер. Слање и примање података од клијента се остварује преко сокета, метода *accept* креира објекат класе *Socket*. Клијент шаље захтев за извршење неке од СО до одговарајуће нити (коју смо назвали *ThreadClient*), која је повезана са тим клијентом. *ThreadClient* прима захтев и даље га преусмерава до класа које су одговорне за извршење СО. Након извршења СО резултат се враћа до апликационе логике, односно до класе *ThreadClient* на серверској страни која тај резултат шаље назад до клијента путем сокета.

4.5 Пословна логика

4.5.1 Пројектовање понашања софтверског система (системске операције)

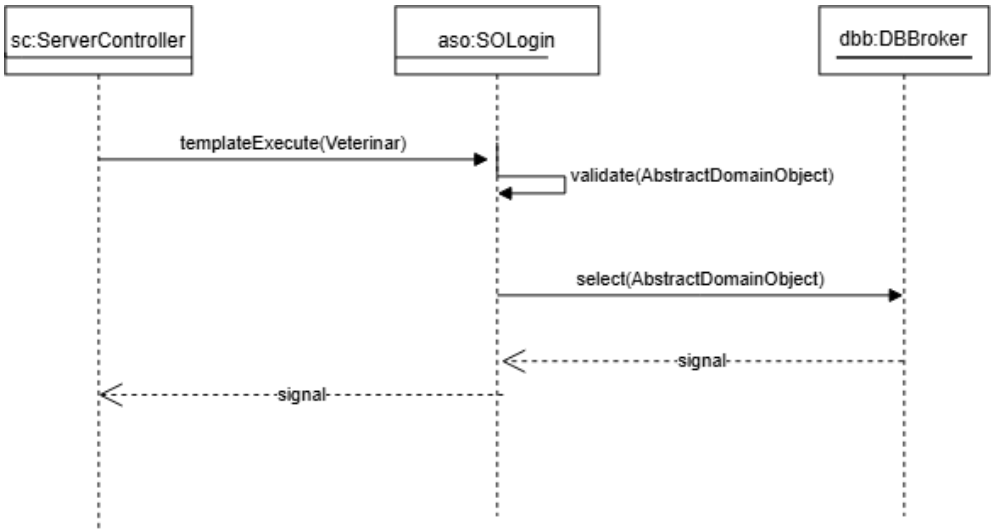
За сваки од претходно дефинисаних уговора правимо системску операцију, што заправо представља пројектовање понашања. Класа *AbstractSO* која представља апстрактну класу која садржи методу *templateExecute*, која представља шаблон извршавања сваке операције над базом података, а као параметар прима објекат класе *AbstractDomainObject*. У тој методи се позивају методе *validate* и *execute*, које су апстрактне и које ће свака класа системске операције имплементирати. Након тога се позива метода *commit* која узима објекат *Connection*, класе *DBBroker* и позива њену методу *commit*. Провера предуслова се извршава на клијентској страни уколико постоји, а постуслови се очитавају у оквиру *Response* објекта који шаље сервер клијенту и на основу кога клијент закључује да ли је операција успешно извршена на серверској страни или је дошло до грешке.

За сваку системску операцију треба направити концептуална решења која су директно повезана са логиком проблема.

За сваки уговор пројектује се концептуално решење.

Уговор UG1: PrijaviVeterinar

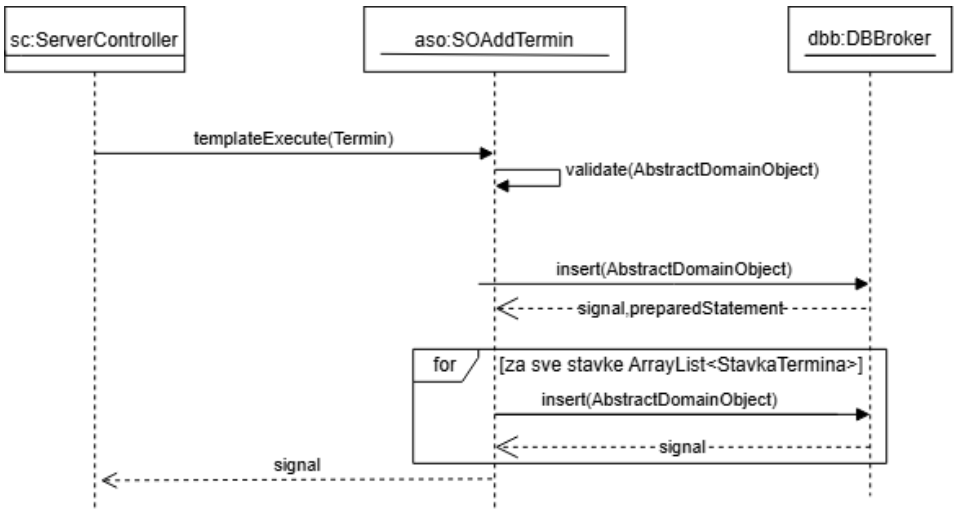
Операција: PrijavaVeterinar (*Korisnickolme, Sifra*):signal;
Веза са СК: СК8
Предуслови:
Постуслови: *Ветеринар је пријављен на систем.*



Слика 57. Уговор UG1: PrijavaVeterinar

Уговор UG2: UbaciTermin

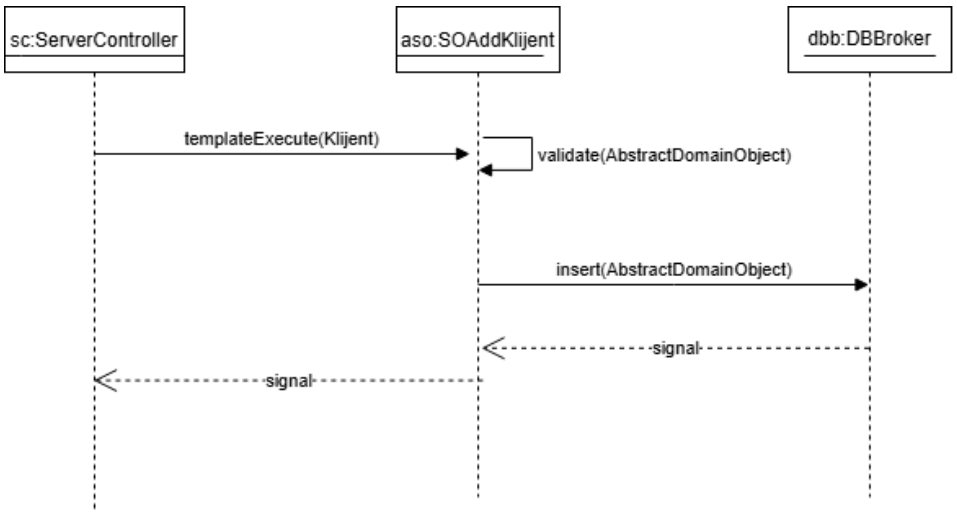
Операција: *UbaciTermin(Termin)*:signal;
Веза са СК: СК1
Предуслови: *Структурна и вредносна ограничење над објектом класе Термин и Ставка термина морају бити задовољена.*
Постуслови: *Направљен је нови објекат класе Термин.*



Слика 58. Уговор UG2: UbaciTermin

Уговор UG3: UbaciKlijent

Операција *UbaciKlijent(Klijent)*:signal;
Веза са СК: СК4
Предуслови: *Структурна и вредносна ограничење над објектом класе Клијент морају бити задовољена.*
Постуслови: *Направљен је нови објекат класе Клијент.*



Слика 59. Уговор UG3: UbaciKlijent

Уговор UG4: PromeniKlijent

Операција: ***PromeniKlijent(Klijent)***:signal;
Веза са СК: СК4, СК6
Предуслови: *Структурна и вредносна ограничење над објектом класе Клијент морају бити задовољена.*
Постуслови: *Објекат класе Клијент је промењен.*



Слика 60. Уговор UG4: PromeniKlijent

Уговор UG5: ObrisiKlijent

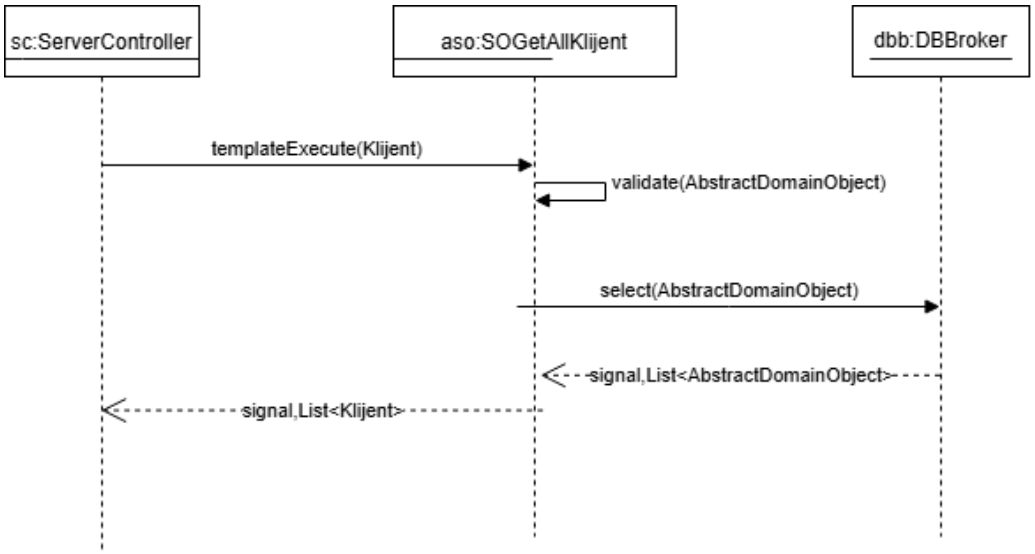
Операција: ***ObrisiKlijent(Klijent***):signal;
Веза са СК: СК7
Предуслови: *Структурна ограничења над објектом класе Клијент морају бити задовољена.*
Постуслови: *Објекат класе Клијент је обрисан.*



Слика 61. Уговор UG5: ObrisiKlijent

Уговор UG6: PretraziKlijent

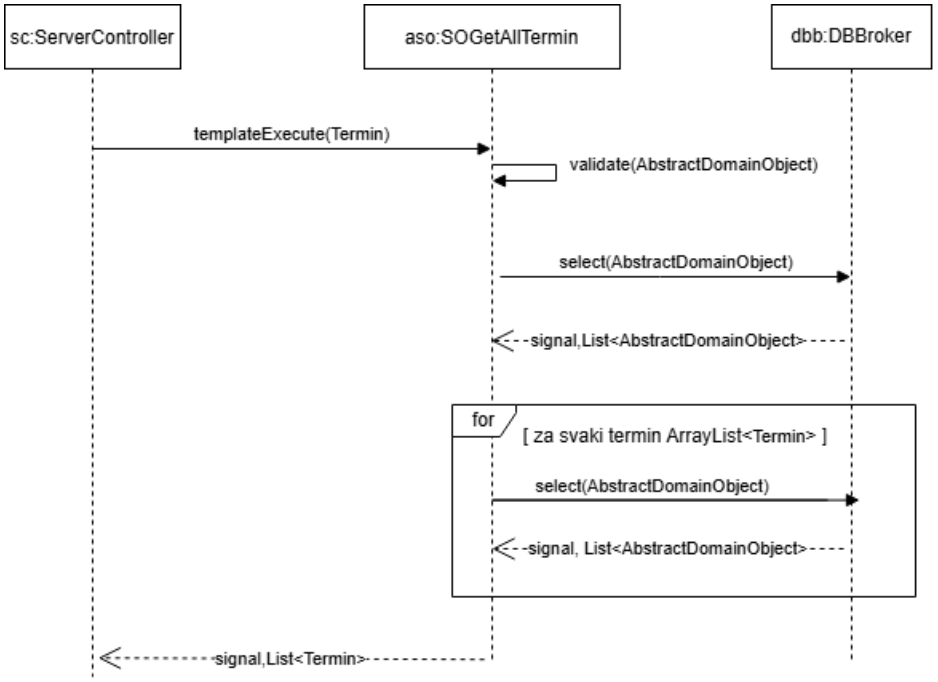
Операција: ***PretraziKlijent(Klijent)***:signal;
Веза са СК: СК5, СК6, СК7
Предуслови:
Постуслови: *Пронађен је тражени објекат класе Клијент.*



Слика 62. Уговор UG6: PretraziKlijent

Уговор UG7: vratiListuTermin

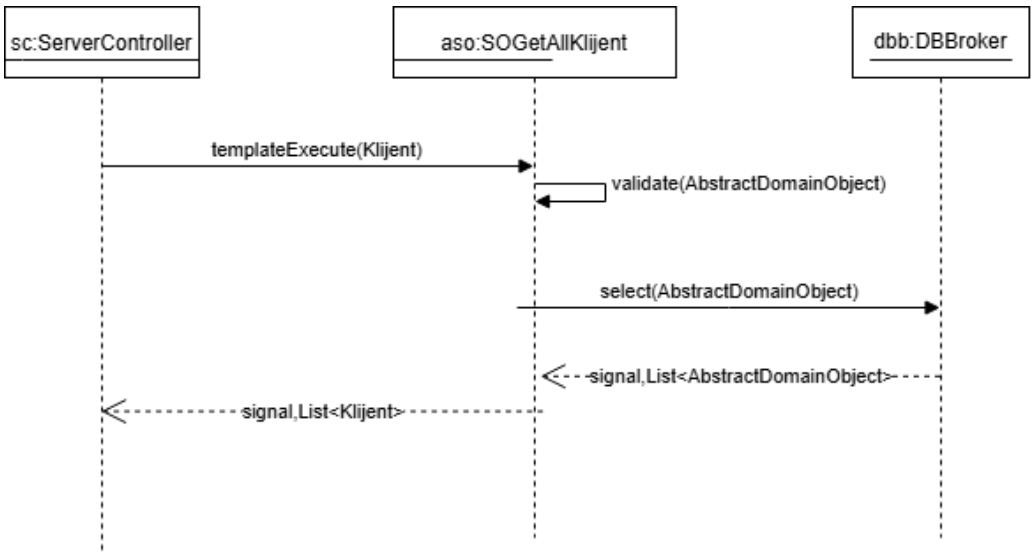
Операција:vratiListuTermin(KriterijumTermin, *List<Termin>*):signal;
Веза са СК: СК2
Предуслови:
Постуслови: *Пронађена је листа тражених објеката класе Термин.*



Слика 63. Уговор UG7: vratiListuTermin

Уговор UG8: vratiListuSviKlijent

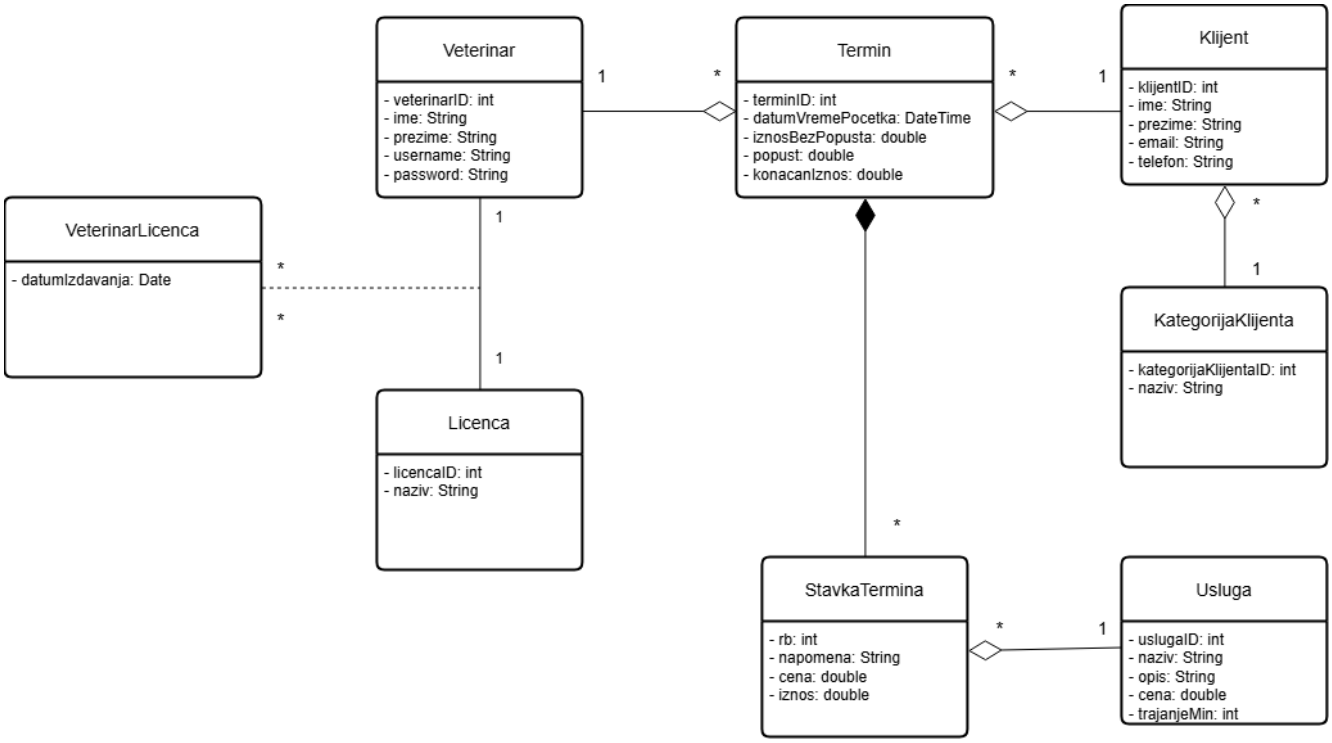
Операција: vratiListuKlijent(*List<Klijent>*):signal;
Веза са СК: СК1, СК3
Предуслови:
Постуслови: *Пронађена је листа свих објеката класе Клијент.*



Слика 64. Уговор UG8: vratiListuSviKlijent

4.5.2 Пројектовање структуре софтверског система

На основу концептуалних класа креирају се софтверске класе.



Слика 65. Концептуални модел ветеринарске ординације

Идентификоване су следеће класе:

- Veterinar

```
public class Veterinar extends AbstractDomainObject {

    private int veterinarID;
    private String ime;
    private String prezime;
    private String username;
    private String password;

    public Veterinar() {
    }

    public Veterinar(int veterinarID, String ime, String prezime, String username, String password) {
        this.veterinarID = veterinarID;
        this.ime = ime;
        this.prezime = prezime;
        this.username = username;
        this.password = password;
    }
```

Слика 66. Класа Veterinar

- VeterinarLicenca

```
public class VeterinarLicenca extends AbstractDomainObject {

    private Licenca licenca;
    private Veterinar veterinar;
    private Date datumIzdavanja;

    public VeterinarLicenca(Licenca licenca, Veterinar veterinar, Date datumIzdavanja) {
        this.licenca = licenca;
        this.veterinar = veterinar;
        this.datumIzdavanja = datumIzdavanja;
    }

    public VeterinarLicenca() {
    }
```

Слика 67. Класа VeterinarLicenca

- Klijent

```
public class Klijent extends AbstractDomainObject {

    private int klijentID;
    private String ime;
    private String prezime;
    private String email;
    private String telefon;
    private KategorijaKlijenta kategorijaKlijenta;

    @Override
    public String toString() {
        return ime + " " + prezime + " (Kategorija klijenta: " + kategorijaKlijenta + ")";
    }

    public Klijent(int klijentID, String ime, String prezime, String email, String telefon, KategorijaKlijenta kategorijaKlijenta) {
        this.klijentID = klijentID;
        this.ime = ime;
        this.prezime = prezime;
        this.email = email;
        this.telefon = telefon;
        this.kategorijaKlijenta = kategorijaKlijenta;
    }

    public Klijent() {
    }
```

Слика 68. Класа Klijent

- KategorijaKlijenta

```
public class KategorijaKlijenta extends AbstractDomainObject {

    private int kategorijaKlijentaID;
    private String naziv;

    public KategorijaKlijenta() {
    }

    public KategorijaKlijenta(int kategorijaKlijentaID, String naziv) {
        this.kategorijaKlijentaID = kategorijaKlijentaID;
        this.naziv = naziv;
    }
```

Слика 69. Класа KategorijaKlijenta

- Termin

```
public class Termin extends AbstractDomainObject {

    private int terminID;
    private Date datumVremePocetka;
    private double iznosBezPopusta;
    private double popust;
    private double konacanIznos;
    private Veterinar veterinar;
    private Klijent klijent;
    private ArrayList<StavkaTermina> stavkeTermina;
    //napravila sam nova polja zbog serverske forme i modela tabele, ona ne idu u konstruktore
    private int filterVeterinarID = -1;
    private String tipBrojanja = null; // "UKUPNO", "BUDUCI", "DANAS"

    public Termin() {
    }

    public Termin(int terminID, Date datumVremePocetka, double iznosBezPopusta, double popust,
            double konacanIznos, Veterinar veterinar, Klijent klijent, ArrayList<StavkaTermina> stavkeTermina) {
        this.terminID = terminID;
        this.datumVremePocetka = datumVremePocetka;
        this.iznosBezPopusta = iznosBezPopusta;
        this.popust = popust;
        this.konacanIznos = konacanIznos;
        this.veterinar = veterinar;
        this.klijent = klijent;
        this.stavkeTermina = stavkeTermina;
    }
```

Слика 70. Класа Termin

- Usluga

```
public class Usluga extends AbstractDomainObject {

    private int uslugaID;
    private String naziv;
    private String opis;
    private double cena;
    private int trajanjeMin;

    public Usluga() {
    }

    public Usluga(int uslugaID, String naziv, String opis, double cena, int trajanjeMin) {
        this.uslugaID = uslugaID;
        this.naziv = naziv;
        this.opis = opis;
        this.cena = cena;
        this.trajanjeMin = trajanjeMin;
    }
```

Слика 71. Класа Usluga

- StavkaTermina

```
public class StavkaTermina extends AbstractDomainObject {

    private Termin termin;
    private int rb;
    private Usluga usluga;
    private String napomena;
    private double cena;
    private double iznos;

    public StavkaTermina() {
    }

    public StavkaTermina(Termin termin, int rb, Usluga usluga,
            String napomena, double cena, double iznos) {
        this.termin = termin;
        this.rb = rb;
        this.usluga = usluga;
        this.napomena = napomena;
        this.cena = cena;
        this.iznos = iznos;
    }
```

Слика 72. Класа StavkaTermina

- Licenca

```
public class Licenca extends AbstractDomainObject {

    private int licencaID;
    private String naziv;

    @Override
    public String toString() {
        return naziv;
    }

    public Licenca(int licencaID, String naziv) {
        this.licencaID = licencaID;
        this.naziv = naziv;
    }

    public Licenca() {
    }
```

Слика 73. Класа Licenca

Поред њих додате су и следеће класе:

- AbstractDomainObject - апстрактна класа које све доменске класе наслеђују.

```
public abstract class AbstractDomainObject implements Serializable {

    public abstract String nazivTabele();
    public abstract String alijas();
    public abstract String join();
    public abstract ArrayList<AbstractDomainObject> vratiListu(ResultSet rs) throws SQLException;
    public abstract String koloneZaInsert();
    public abstract String vrednostiZaInsert();
    public abstract String vrednostiZaUpdate();
    public abstract String uslov();
    public abstract String dodatniUslov();
    public abstract String orderBy();

}
```

Слика 74. Класа AbstractDomainObject

- Operation – интерфејс који садржи све операције које се шаљу од клијента серверу.

```
public interface Operation {

    public static final int LOGIN = 0;
    public static final int LOGOUT = 1;

    public static final int ADD_KLIJENT = 2;
    public static final int DELETE_KLIJENT = 3;
    public static final int UPDATE_KLIJENT = 4;
    public static final int GET_ALL_KLIJENT = 5;

    public static final int ADD_USLUGA = 6;
    public static final int DELETE_USLUGA = 7;
    public static final int UPDATE_USLUGA = 8;
    public static final int GET_ALL_USLUGA = 9;

    public static final int ADD_TERMIN = 10;
    public static final int DELETE_TERMIN = 11;
    public static final int UPDATE_TERMIN = 12;
    public static final int GET_ALL_TERMIN = 13;

    public static final int ADD_KATEGORIJA_KLIJENTA = 14;
    public static final int DELETE_KATEGORIJA_KLIJENTA = 15;
    public static final int UPDATE_KATEGORIJA_KLIJENTA = 16;
    public static final int GET_ALL_KATEGORIJA_KLIJENTA = 17;

}
```

Слика 75. Interface Operation

- Request - служи за слање објекта од клијента ка серверу. Садржи један int атрибут који представља операцију која треба да се изврши и један *Object* атрибут који представља параметар који се шаље серверу да би извршио ту операцију.

```
public class Request implements Serializable {

    private int operation;
    private Object data;

    public Request() {
    }

    public Request(int operation, Object data) {
        this.operation = operation;
        this.data = data;
    }
```

Слика 76. Класа Request

- Response - служи за слање објекта од сервера ка клијенту. Садржи један *Object* атрибут који представља резултат извршене операције, један *Exception* атрибут који представља изузетак који се можда десио.

```
public class Response implements Serializable {

    private Object data;
    private Exception exc;
    private ResponseStatus responseStatus;

    public Response() {
    }

    public Response(Object data, Exception exc, ResponseStatus responseStatus) {
        this.data = data;
        this.exc = exc;
        this.responseStatus = responseStatus;
    }
```

Слика 77. Класа Response

4.5.3 Пројектовање складишта података

На основу релационог модела и ограничења пројектоване су табеле базе података које користи наш софтверски систем:

Column Name	Data Type	Length	Default	PK?	Not Null?	Unsigned?	Auto Incr?	Zerofill?	On Update	Comment
VeterinarID	bigint	10		☑	☑	☑	☑	☐	☐	
Ime	varchar	50		☐	☑	☐	☐	☐	☐	
Prezime	varchar	50		☐	☑	☐	☐	☐	☐	
Username	varchar	50		☐	☑	☐	☐	☐	☐	
Password	varchar	100		☐	☑	☐	☐	☐	☐	
				☐	☐	☐	☐	☐	☐	

Слика 78. Табела Ветеринар

Column Name	Data Type	Length	Default	PK?	Not Null?	Unsigned?	Auto Incr?	Zerofill?	On Update	Comment
VeterinarID	bigint	10		☑	☑	☑	☐	☐	☐	
LicencaID	bigint	10		☑	☑	☑	☐	☐	☐	
DatumIzdavanja	date			☑	☑	☐	☐	☐	☐	
				☐	☐	☐	☐	☐	☐	

Слика 79. Табела Ветеринар-Лиценца

Column Name	Data Type	Length	Default	PK?	Not Null?	Unsigned?	Auto Incr?	Zerofill?	On Update	Comment
KlijentID	bigint	10		☑	☑	☑	☑	☐	☐	
Ime	varchar	50		☐	☑	☐	☐	☐	☐	
Prezime	varchar	50		☐	☑	☐	☐	☐	☐	
Email	varchar	100		☐	☑	☐	☐	☐	☐	
Telefon	varchar	40		☐	☑	☐	☐	☐	☐	
KategorijaKlijent…	bigint	10		☐	☑	☑	☐	☐	☐	
				☐	☐	☐	☐	☐	☐	

Слика 80. Табела Клијент

Column Name	Data Type	Length	Default	PK?	Not Null?	Unsigned?	Auto Incr?	Zerofill?	On Update	Comment
KategorijaKlijent…	bigint	10		☑	☑	☑	☑	☐	☐	
Naziv	varchar	50		☐	☑	☐	☐	☐	☐	
				☐	☐	☐	☐	☐	☐	

Слика 81. Табела Категорија клијента

Column Name	Data Type	Length	Default	PK?	Not Null?	Unsigned?	Auto Incr?	Zerofill?	On Update	Comment
TerminID	bigint	10		☑	☑	☑	☑	☐	☐	
DatumVremePocetka	datetime			☐	☑	☐	☐	☐	☐	
IznosBezPopusta	decimal	10,2		☐	☑	☐	☐	☐	☐	
Popust	decimal	5,2		☐	☑	☐	☐	☐	☐	
KonacanIznos	decimal	10,2		☐	☑	☐	☐	☐	☐	
VeterinarID	bigint	10		☐	☑	☑	☐	☐	☐	
KlijentID	bigint	10		☐	☑	☑	☐	☐	☐	
				☐	☐	☐	☐	☐	☐	

Слика 82. Табела Термин

Column Name	Data Type	Length	Default	PK?	Not Null?	Unsigned?	Auto Incr?	Zerofill?	On Update	Comment
UslugaID	bigint	10		☑	☑	☑	☑	☐	☐	
Naziv	varchar	100		☐	☑	☐	☐	☐	☐	
Opis	varchar	255		☐	☑	☐	☐	☐	☐	
Cena	decimal	10,2		☐	☑	☐	☐	☐	☐	
TrajanjeMin	int	7		☐	☑	☐	☐	☐	☐	
				☐	☐	☐	☐	☐	☐	

Слика 83. Табела Услуга

	Column Name	Data Type	Length	Default	PK?	Not Null?	Unsigned?	Auto Incr?	Zerofill?	On Update	Comment
☐	TerminID	bigint	10		☑	☑	☑	☐	☐	☐	
☐	Rb	int	7		☑	☑	☐	☐	☐	☐	
☐	UslugaID	bigint	10		☐	☑	☑	☐	☐	☐	
☐	Napomena	varchar	200		☐	☑	☐	☐	☐	☐	
☐	Cena	decimal	10,2		☐	☑	☐	☐	☐	☐	
☐	Iznos	decimal	10,2		☐	☑	☐	☐	☐	☐	
☐					☐	☐	☐	☐	☐	☐	

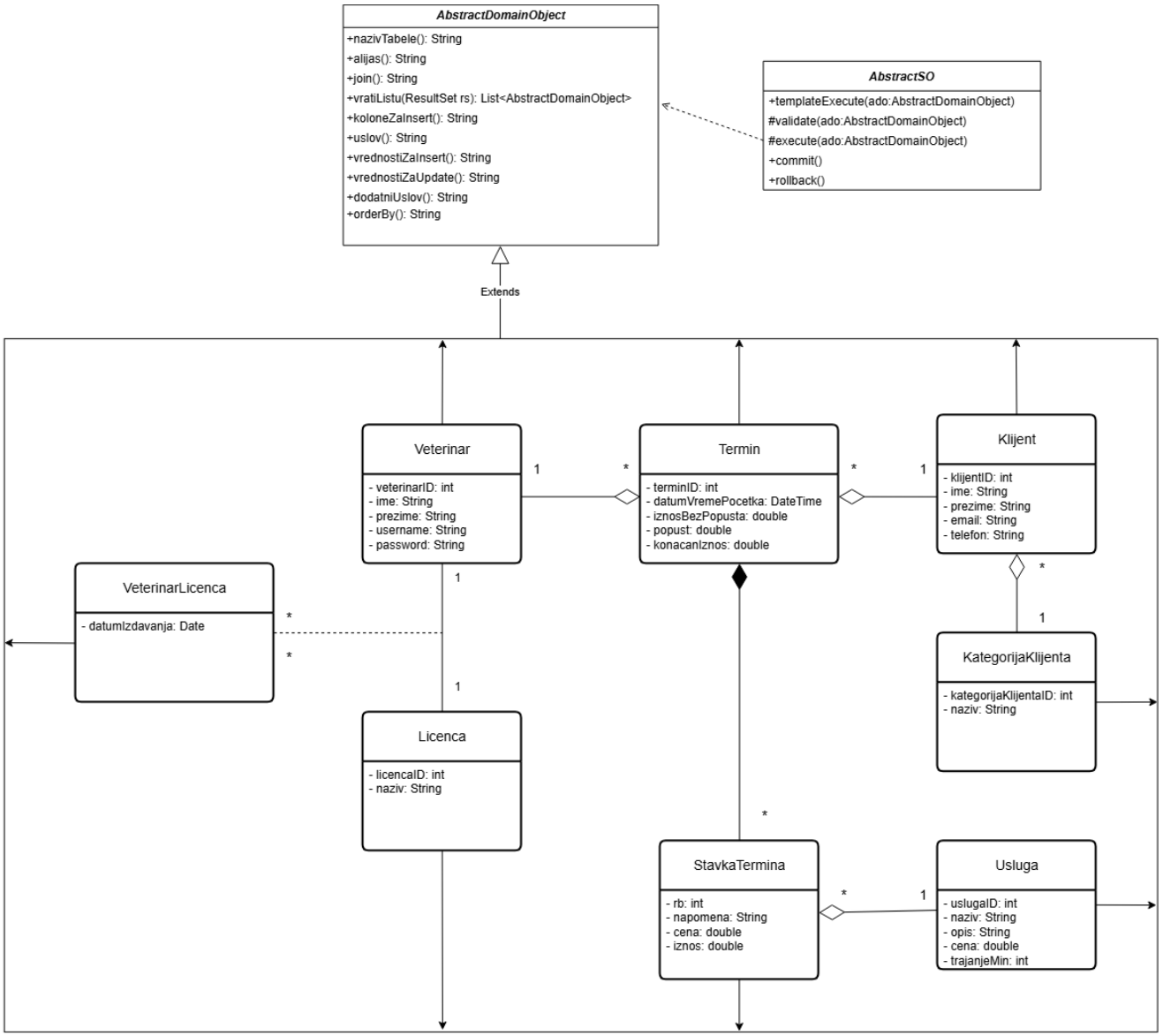
Слика 84. Табела СтавкаТермина

	Column Name	Data Type	Length	Default	PK?	Not Null?	Unsigned?	Auto Incr?	Zerofill?	On Update	Comment
☐	LicencaID	bigint	10		☑	☑	☑	☑	☐	☐	
☐	Naziv	varchar	100		☐	☑	☐	☐	☐	☐	
☐					☐	☐	☐	☐	☐	☐	

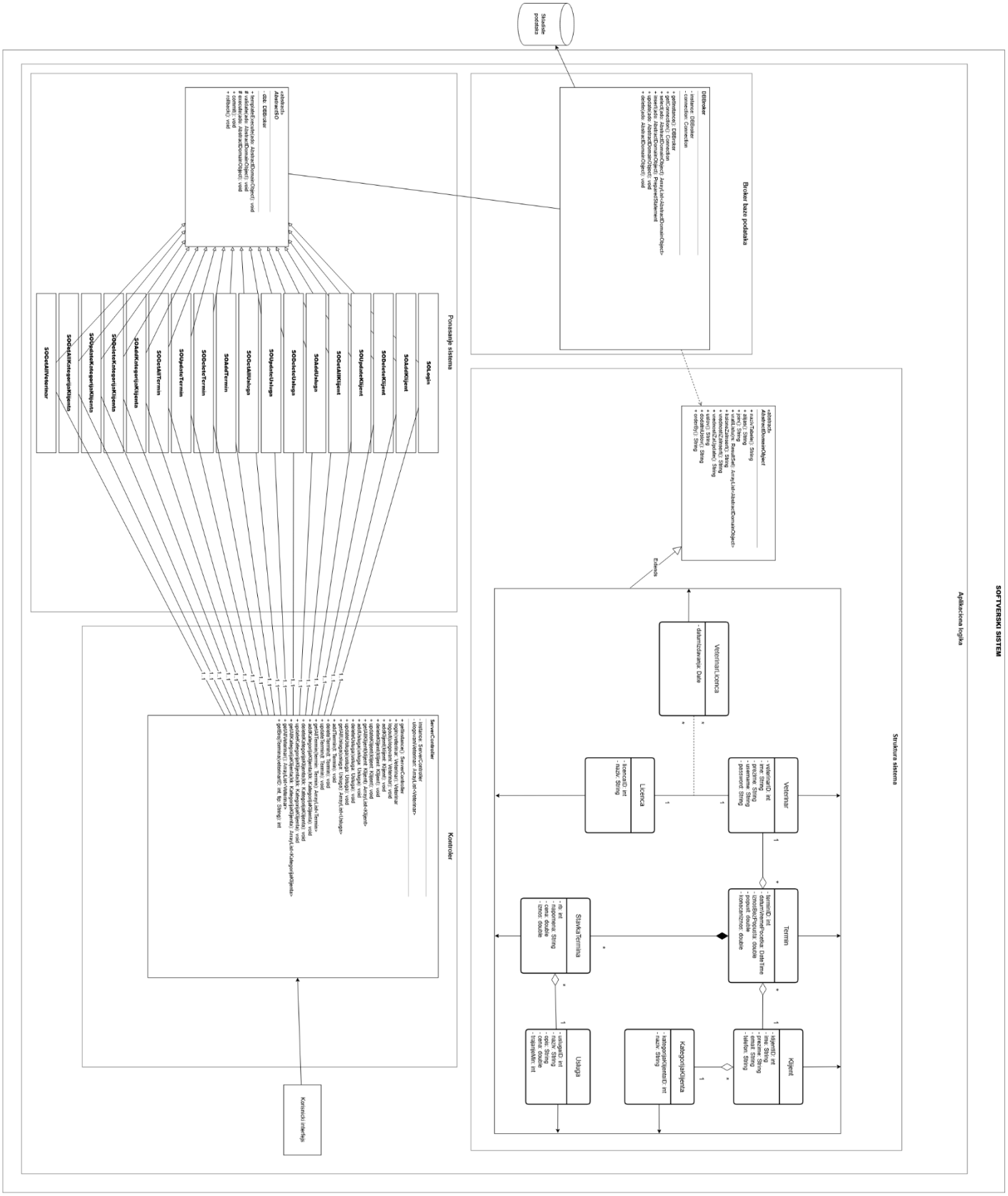
Слика 85. Табела Лиценца

За комуникацију са базом података правимо генеричку класу *AbstractSO* која има методе за валидацију и извршавања трансакције, које су апстрактне и које ће имплементирати свака класа системске операције која ће се извршавати, као и методе за потврђивање и поништавање трансакције. Она се служи класом *DBBroker* која је имплементирана помоћу *Singleton* патерна и која параметре за повезивање на базу података чита из текстуалног фајла који садржи све потребне параметре и помоћу ње наша генеричка класа добија конекцију на базу података.

Као резултат пројектовања класе *AbstractSO* и доменских објеката добијамо следећи дијаграм класа:



Слика 86. Дијаграм класа добијен након пројектовања доменских класа и AbstractSOкласе

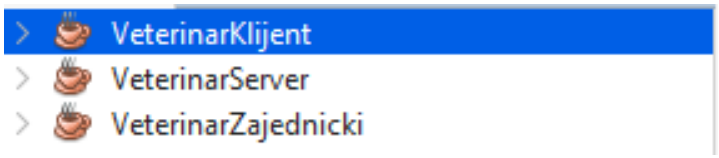


Слика 87. Коначна архитектура софтверског система

5. Имплементација

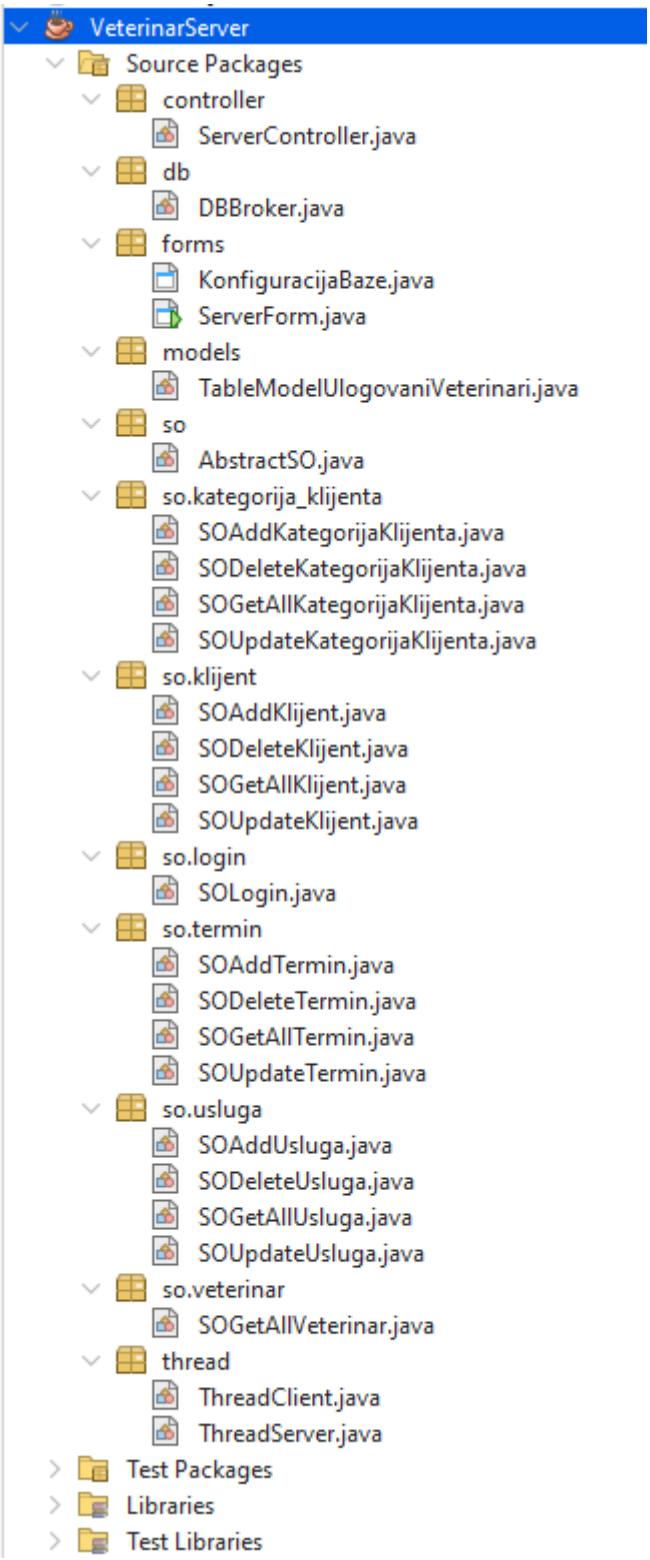
Софтверски систем је развијан у програмском језику “Java”, развојно окружење Apache NetBeans. Као систем за управљање базом података коришћен је MySQL.

Организација пројеката је приказана на следећој слици.



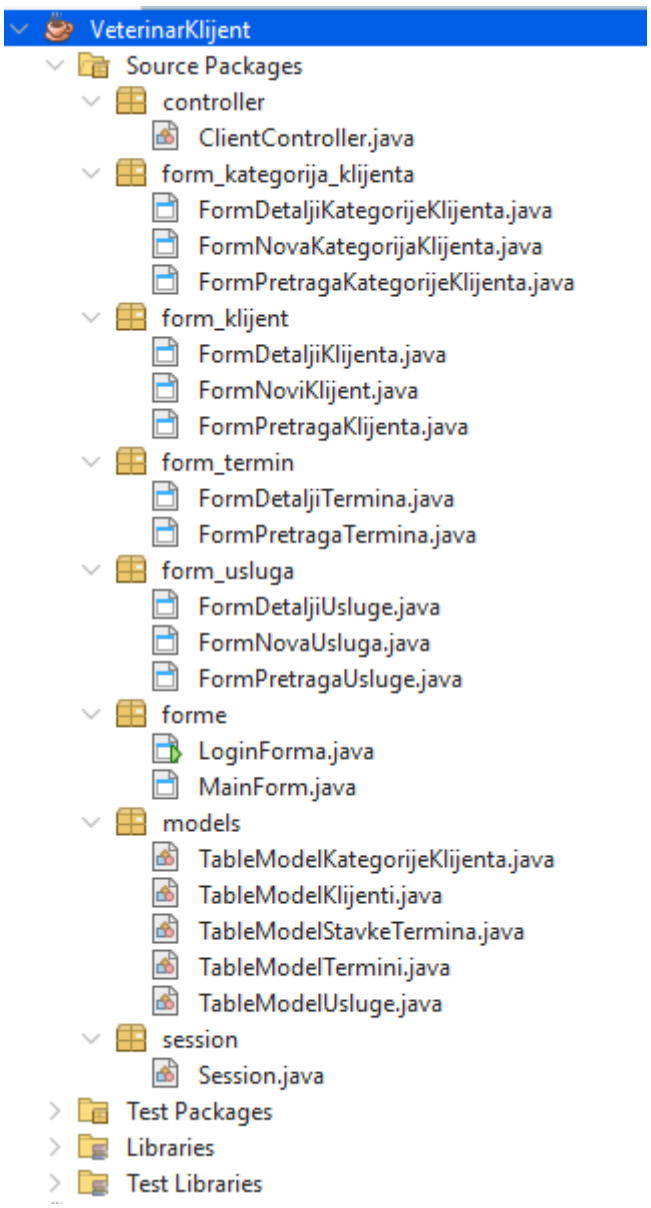
Слика 88. Организација пројекта

Пројекат VeterinarServer садржи нити за покретање комуникације са клијентом, контролера, генеричку класу за базу података и генерисање конекције на базу података и форму за измену конфигурације базе.



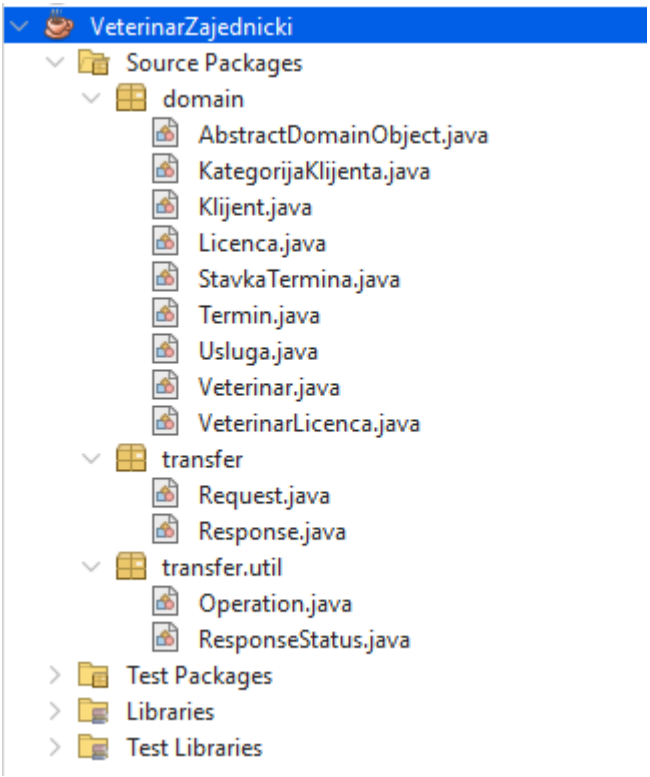
Слика 89. Серверски део

Пројекат VeterinarKlijent садржи форме на којима ветеринар ради и сокет који служи за комуникацију са сервером.



Слика 90. Клијентски део

Пројекат VeterinarZajednicki садржи заједничке класе које користе клијент и сервер пројекти. То су доменске класе, класе *Request* и *Response* и интерфејс *Operation* и енум *ResponseStatus*.



Слика 91. Заједнички део

6. Тестирање

Фаза тестирања представља један од најважнијих корака у процесу развоја софтвера, будући да омогућава проверу да ли систем заиста испуњава све дефинисане захтеве и да ли је спреман за примену у стварном окружењу. Тестирање израђене апликације за управљање услугама у ветеринарској ординацији спроведено је систематично, у више корака, како би се проверила поузданост, стабилност и исправност система у различитим сценаријима коришћења.

Пре свега, посебна пажња посвећена је провери поступка пријаве на систем. Тестирано је понашање апликације приликом уноса исправних корисничких података, али и приликом уноса нетачног корисничког имена или лозинке, као и покушаја пријаве са празним пољима. Проверено је да ли систем у тим ситуацијама исправно пријављује грешку и онемогућава приступ неовлашћеним корисницима, чиме се потврђује да апликација исправно спречава истовремену пријаву истог ветеринара, као и приступ са непостојећим креденцијалима.

Након тога, сваки случај коришћења дефинисан у фази анализе тестиран је засебно. За сваки од њих најпре су уношени подаци који представљају исправне, очекиване вредности, како би се утврдило да ли систем даје тачан резултат у стандардним условима рада. Затим су свесно коришћени и неисправни подаци, као и граничне вредности уноса, са циљем да се провери како апликација реагује у непредвиђеним ситуацијама и да ли су имплементирани механизми за валидацију довољно поуздани. На пример, приликом евидентирања новог клијента тестиране су ситуације са недостајућим или празним пољима, како би се потврдило да апликација не дозвољава чување записа све док сви обавезни подаци нису унети на исправан начин. Слично томе, приликом уноса нове услуге проверено је понашање система када се у поља предвиђена за цену и трајање унесу словни или неисправно форматирани подаци, чиме је потврђено да апликација правилно препознаје погрешан тип уноса и спречава наставак рада.
Посебна пажња посвећена је тестирању рада са терминима, будући да термин представља основу за формирање рачуна за пружене ветеринарске услуге. Проверено је да ли систем исправно обрађује унос датума и времена у очекиваном формату, као и да ли правилно пријављује грешку када се датум унесе у неисправном облику. Такође је тестирано додавање ставки, односно услуга, у оквиру једног термина, при чему је потврђено да апликација онемогућава двоструко додавање исте услуге истом термину.

Поред функционалног тестирања, посебно је проверена исправност рачунских операција приликом формирања коначног износа рачуна. Испитано је да ли се укупан износ услуга правилно израчунава, као и да ли се одобрени попуст исправно примењује у зависности од категорије којој клијент припада. На тај начин потврђено је да обрачун за основне, сребрне и златне клијенте одговара очекиваним вредностима, што је од кључног значаја за поузданост целог система.

Тестиран је и кориснички интерфејс, при чему се проверавало да ли је довољно прегледан и интуитиван, односно да ли корисник може без додатних објашњења да разуме на који начин да користи систем. Циљ је био да свакодневни рад ветеринарског особља буде што једноставнији и бржи.

На крају, резултати тестирања показали су да апликација задовољава све дефинисане захтеве и да је спремна за употребу у реалном окружењу. Иако су уочени поједини мањи недостаци, они не представљају критичне грешке и могу се отклонити у наредним

фазама развоја. На основу спроведеног тестирања може се закључити да је систем стабилан, поуздан и практичан за коришћење у свакодневном раду ветеринарске ординације.

7. Закључак

Реализација пројекта система за управљање услугама у ветеринарској ординацији показала је значај примене методичног приступа у развоју софтвера. Кроз све фазе рада, од прикупљања захтева и анализе, преко пројектовања и имплементације, па све до тестирања, настојала сам да применим знања стечена током студија и прилагодим их конкретном проблему. Резултат овог процеса је функционална клијент-сервер апликација која омогућава ефикасну евиденцију клијената, услуга и термина, као и једноставније управљање самим процесом формирања и издавања рачуна за пружене ветеринарске услуге.

Систем је осмишљен тако да у пракси може у великој мери да олакша свакодневни рад запослених у ветеринарским ординацијама. Прегледан интерфејс и логична структура података омогућавају лаку примену и оним корисницима који немају претходно искуство у раду са оваквим апликацијама. Посебно је обрађена пажња на аутоматизацију обрачуна, тако да се коначан износ рачуна и одобрени попуст, у зависности од категорије клијента, израчунавају без потребе за ручним рачунањем, чиме се смањује могућност грешке и убрзава рад особља.

Са друге стране, структура система отвара простор за даљи развој, било увођењем нових функционалности, попут аутоматизованих подсетника за наредне прегледе, вођења картона животиња или генерисања извештаја о пословању ординације. Избор Јаве као програмског језика показао се као оправдан, јер је, захваљујући својој објектно-оријентисаној природи и богатој заједници, омогућио јасно моделирање домена и лакше решавање изазова током развоја. Примена клијент-сервер архитектуре додатно је допринела прегледности решења и раздвајању пословне логике од корисничког интерфејса.

Рад на овом пројекту био је прилика да стекнем дубље разумевање процеса развоја апликација у реалном окружењу, али и да се сусретнем са практичним проблемима који настају приликом преласка са теорије на примену. Посебну вредност овог искуства видим у томе што ми је омогућило да повежем више области које сам изучавала током студија, од анализе захтева и моделирања система, до дизајна корисничког интерфејса и програмирања. Сам процес је захтевао креативност, упорност и способност критичког размишљања, а завршетак пројекта ми је донео не само функционално решење, већ и осећај сигурности у сопствене способности за будуће професионалне изазове.

Литература

Oracle. (n.d.). *Java dokumentacija*. https://docs.oracle.com/en/java/

Stack Overflow. (n.d.). *Stack Overflow* [forum]. https://stackoverflow.com/

Tomić, B., Jovanović, J., Milikić, N., Ševarac, Z., & Đurić, D. (2018). *Principi programiranja*. Beograd.

Vlajić, S. (2020). *Projektovanje softvera* [skripta]. Beograd.